

Quantum Algorithms For Iterations Of Fractal Type Functions with Exponential Sums: Disproving or Verifying the Collatz Conjecture

M.Chowdhury

Draft version

Written 3rd May 2025^{*} Revised 25th September 2026^{*}

(Dates of server time stamps)*

Abstract

The Collatz Conjecture (CC) asserts that every positive integer eventually reaches the cycle $1 \rightarrow 4 \rightarrow 2 \rightarrow 1$ under the map $x \mapsto x/2$ (x even), $x \mapsto 3x + 1$ (x odd). Starting with a new reformulation of the CC using

$$x_{n+1} = qx_n + wI\left(\sum_{K=0}^U \left(-\frac{1}{2}2^K\left(\frac{1}{4}\right)^P\right)\left(e^{\frac{i}{2^K}\pi x_n} - 1\right)\left(e^{-\frac{i}{2^K}\pi x_n} + e^{\frac{i}{2^K}\pi x_n} - 2\right)^P\right)$$

We present a division-free reformulation of CC and its $qx + w$ generalisation in the order it was actually developed: from a general analytic (exponential-sum) representation of a bitwise operator $\text{LSB}(x)$ ($\text{LSB}(x)$ is a function that has the formal definition given in this paper) valid for arbitrary odd q, w , where w maybe positive or negative that multiplies $\text{LSB}(x)$, through its exact character-sum and closed-form-sum counterparts, to the general algebraic map $g_{q,w}(x) = qx + w\text{LSB}(x)$ and the exact dichotomy governing exactly which (q, w) admit a clean, single-bit convergence test, specialising last to the classical case $q = 3, w = 1$. On that classical case we give the first quantum algorithm, with a concrete oracle, that verifies the Collatz Conjecture and its natural generalisation, on a finite interval faster, in query complexity, than any known classical method, using Grover's algorithm and quantum amplitude estimation. If a counterexample exists within the searched interval, the algorithm finds it ; exhibiting an explicit integer that enters a cycle other than $1 \rightarrow 4 \rightarrow 2 \rightarrow 1$ and thereby disproving CC ; quadratically faster, in query count, than the fastest classical search that could find the same counterexample; if none is found, this only verifies CC on that interval and does not itself disprove or prove the conjecture. We also state a strengthening of CC, the *logarithmic stopping-time conjecture* CC' , and show how the algorithm's four possible outcomes on a finite run should be interpreted with respect to CC and CC' .

1 Introduction

The idea of for a quantum algorithm to the Generalised Collatz Conjecture or Collatz Conjecture/ $3x+1$ problem, below, was expressed in our 2023 paper [9] in the conclusion section of [9]. The Collatz function is

$$f(x) = \begin{cases} x/2 & x \equiv 0 \pmod{2} \\ 3x + 1 & x \equiv 1 \pmod{2}, \end{cases} \quad (200)$$

and CC asserts that for every positive integer x_0 , some iterate $f^{(t)}(x_0)$ lands in $\{1, 2, 4\}$. CC has been verified for all x_0 below roughly 2^{68} by classical computation, with no counterexample and no

new cycle found [1]. No classical algorithm can verify a range $[1, N]$ in better than $O(N)$ time in the worst case, since (absent structural insight) every starting value must be tried.

With our reformulation of the CC then Grover's algorithm can be applied to it: because the above is searching an unstructured space of size N for a marked item, with a quadratic speed-up to $O(\sqrt{N})$ queries. We applying it to CC using our two following ideas: (1) an oracle that recognises a counterexample using a bounded number of iterations, and (2) a reversible arithmetic circuit that computes one Collatz step without a general division circuit. Section 3 builds (2) from the ground up in the order it was discovered , most general construction first, classical case last, and proves along the way exactly which part of the construction is special to $3x + 1$ rather than generic to $qx + w$; Section 5 gives the oracle and algorithm built on top of the classical specialisation.

A note on scope

This algorithm verifies CC (and $qx + w$) on a *chosen finite interval* $[1, 2^m]$. Doing so faster does not bring us closer to deciding CC in general: any finite verification, classical or quantum, leaves the infinitude of untested integers untouched. What the construction offers is a quadratic reduction in the query count needed to extend the verified range, contingent on a fault-tolerant quantum computer large enough to run the arithmetic circuits described in Section 3 ; which is not available today even for modest m . In this paper we use the widely use assumption/conjecture that a generic quantum circuit can not be simulated on a classical only computer in polynomial time.

2 The Logarithmic Stopping-Time Conjecture

Let $\tau(x)$ be the stopping time of x : the least t with $f^{(t)}(x) \in \{1, 2, 4\}$.

Conjecture 1 (CC). *Every positive integer has a finite stopping time.*

Conjecture 2 (CC'). *There exists a universal constant $\eta > 0$ such that $\tau(x) \leq \lfloor \eta \log x \rfloor$ for every positive integer x .*

Clearly $CC' \Rightarrow CC$; the converse is open. So CC' is a strengthening of CC, not a restatement of it, and a counterexample to CC' need not be a counterexample to CC ;it may just mean η was chosen too small. This is the reason the algorithm below has four outcomes rather than two: it is simultaneously probing CC and CC' using a single candidate constant η_0 .

3 From the General Construction to the Classical Case

3.1 Two related functions

Write x in binary. Two closely related quantities matter here, and keeping them distinct is the whole point of this section:

Definition 1. $\nu(x)$ is the position of the lowest set bit of x (its 2-adic valuation): $\nu(x) = k$ where $2^k \mid x$ and $2^{k+1} \nmid x$. For odd x , $\nu(x) = 0$.

Definition 2. $LSB(x) = 2^{\nu(x)}$ is the value of the lowest set bit – equivalently the largest power of two dividing x . For odd x , $LSB(x) = 1$. One disambiguation is worth stating plainly here: in much of the computer-science literature “LSB” instead denotes the single bit at position 0 (i.e. $x \bmod 2$, either 0 or 1). That is not what is meant in this paper: $LSB(x)$ here is the numeric value of the lowest set bit;a power of two that can be arbitrarily large ;not the bit itself.

So in this paper the $\text{LSB}(x)$ function has a precise formal definition given by the above. So $\text{LSB}(10) = 2$ (since $10 = 8 + 2$), $\text{LSB}(6) = 2$, $\text{LSB}(7) = 1$. Every construction below, consequently from the identity with exponential sums, is built from LSB , not ν : using ν in place of LSB drops the $+1$ on every odd input and makes any resulting sequence trivially divergent, since $\nu(x) \geq 0$ contributes almost nothing. LSB instead gives, at the most general level of q, w before any specialisation, a family of maps whose convergence behaviour is provably tied to the Collatz-type conjecture for that (q, w) ; which is what the rest of this section develops, from most general to most specific.

3.2 The general analytic master equation

The first form of this construction to be discovered was not the bitwise one but an analytic, exponential-sum representation of $\text{LSB}(x)$, valid for arbitrary odd q, w since q, w never appear inside the representation of LSB itself; only outside it, exactly as in the algebraic map introduced in Section 3.4 where w maybe positive or negative but we strict to w positive in this paper since it makes the circuitry more efficient as there is no need to implement a negative numbers say with two's complement arithmetic; for example choosing $w = -1$ corresponds to the "3x-1 problem". Encoding divisibility by 2^{K+1} through the 2^{K+1} -th roots of unity, in the spirit of Chamberland's continuous extension of the Collatz map to the real line [3], the general master equation is:

$$x_{n+1} = qx_n + w I \left(\sum_{K=0}^U \left(-\frac{1}{2} 2^K \left(\frac{1}{4} \right)^p \right) \left(e^{\frac{i}{2^K} \pi x_n} - 1 \right) \left(e^{-\frac{i}{2^K} \pi x_n} + e^{\frac{i}{2^K} \pi x_n} - 2 \right)^p \right) \quad (1)$$

where $I(\cdot)$ rounds to the nearest integer, and q, w are arbitrary odd positive integers. This is not needed by the quantum algorithm of Section 5, in the specific implementation there, which uses the exact finite bitwise circuit of Section 3.6 instead; it is recorded here, first, because it is where the construction began, and because it connects to classical analytic-number-theory machinery worth naming precisely (Section 4).

Proposition 1. *Fix $x > 0$ with $\nu(x) = K_0$, and let $U \geq K_0$ be fixed. As $p \rightarrow \infty$ through even values, the sum inside $I(\cdot)$ in (1) converges to $\text{LSB}(x) = 2^{K_0}$, independent of q and w (which do not appear inside the sum).*

Proof. Write $\theta_K = \pi x / 2^K$ and $a_K = e^{i\theta_K}$, so the K -th term is $T_K(p) = -\frac{1}{2} 2^K \left(\frac{1}{4} \right)^p (a_K - 1)(a_K^{-1} + a_K - 2)^p$, and $a_K^{-1} + a_K - 2 = 2 \cos \theta_K - 2 = -4 \sin^2(\theta_K/2)$. At $K = K_0$, $x/2^{K_0}$ is an odd integer, so $\theta_{K_0} = \pi \cdot (\text{odd})$ and $a_{K_0} = -1$ exactly; substituting gives $T_{K_0}(p) = -\frac{1}{2} \cdot 2^{K_0} \cdot \left(\frac{1}{4} \right)^p \cdot (-2) \cdot (-4)^p = 2^{K_0} (-1)^p$, independent of p in magnitude. For $K \neq K_0$ with $K \leq U$, $x/2^K$ is not an odd integer, so $|\sin(\theta_K/2)| < 1$ strictly, and $|T_K(p)| = \frac{1}{2} 2^K |a_K - 1| \cdot 4^{-p} \cdot (4 |\sin(\theta_K/2)|^2)^p \rightarrow 0$ geometrically as $p \rightarrow \infty$. Summing over the fixed, finite range $K = 0, \dots, U$, all terms but T_{K_0} vanish, and for even p , $T_{K_0}(p) = 2^{K_0} = \text{LSB}(x)$ exactly. $\square$

We verified this numerically (agreement with $\text{LSB}(x)$ to six decimal places by $p \approx 100$ for x up to several hundred, with slower convergence for larger x since the discarded terms' decay rate depends on x through θ_K). Note that the surviving term carries a factor $(-1)^p$, so p must be taken *even* (or the result passed through $|\cdot|$) for the limit to be $+\text{LSB}(x)$ rather than $-\text{LSB}(x)$; this parity requirement is necessary and should be stated as part of the construction wherever it is used.

Remark 1 (Directions suggested by, but not established in, this construction). *The complex-valued form above suggests several further questions worth recording, though none is developed into a result here: (i) extending $x_n \in \mathbb{N}$ to $z \in \mathbb{C}$ turns the iteration into a complex dynamical system, and*

asking about its Fatou/Julia structure is a reasonable question, but no such structure is computed or characterised in this paper; (ii) the phase factors $e^{i\theta_K}$ are the right shape to map onto phase-shift gates $R_\theta = \text{diag}(1, e^{i\theta})$, which is a plausible route to an alternative, phase-based circuit for $\text{LSB}(x)$, but we have not built or benchmarked such a circuit here, and it would need to be compared against the exact bitwise circuit of Section 3.6 in gate count before being called an improvement (Section 4.4 gives a sharper, structural reason to expect this could work); (iii) the construction is already stated for general $qx + w$ above; (iv) the individual terms above do isolate the residue class $x \bmod 2^{K+1}$, which is a fair description of what the geometric-decay argument in the proof is doing, but we would not go further than that without a precise claim to check.

Remark 2 (What kind of symmetry is actually being used). Equation (6.7) is built from $\zeta_K = e^{i\pi/2^K}$, a primitive 2^{K+1} -th root of unity, and evaluates ζ_K^x as x varies. This is precisely an additive character of the cyclic group $\mathbb{Z}/2^{K+1}\mathbb{Z}$, the basic object of discrete Fourier analysis on a finite cyclic group, and the proof of Proposition 1 is a concentration/orthogonality argument of exactly that kind: raising $(\cos \theta_K - 1)$ to a large power p suppresses every residue class except the one where $|\sin(\theta_K/2)| = 1$, which is what lets a single term survive in the limit. This is why the construction works, and it is worth naming precisely, because the superficially similar-looking object; the Galois group $\text{Gal}(\mathbb{Q}(\zeta_K)/\mathbb{Q}) \cong (\mathbb{Z}/2^{K+1}\mathbb{Z})^\times$, which permutes ζ_K among its other primitive roots via $\zeta_K \mapsto \zeta_K^j$ for j coprime to 2^{K+1} ; is not used anywhere in the construction: for each fixed K , equation (6.7) varies the exponent x over one fixed root ζ_K , not the choice of root over its Galois conjugates. A genuinely Galois-theoretic question one could ask instead is whether summing over the full Galois orbit of ζ_K , i.e. over every primitive 2^{K+1} -th root, not just the one used here, reveals anything the single-character sum does not (for instance, whether it gives a sharper concentration bound or detects a coarser invariant of x than $\nu_2(x)$). This is a real, answerable question, but it is not the one equation (6.7) answers, and we have not investigated it.

3.3 The general closed-form jump operator, generic q, w

Unrolling $f^n(x)$ for the classical case $q = 3, w = 1$ by repeated substitution, $f^0(x) = x$, $f^1(x) = 3x + \text{LSB}(x)$, $f^2(x) = 3f^1(x) + \text{LSB}(f^1(x))$, and so on, gives a, non-recursive form, . In general odd q, w before any specialisation the non-recursive genral form is as follows; for the general map $f_{q,w}(x) = qx + w\text{LSB}(x)$ with q odd:

Proposition 2. For every $n \geq 0$,

$$f_{q,w}^{(n)}(x) = q^n x + w \sum_{k=0}^{n-1} q^{n-1-k} \text{LSB}\left(f_{q,w}^{(k)}(x)\right). \quad (201)$$

Proof. Induction on n . $n = 0$ gives x , matching the empty sum. Assume the identity at n ; then

$$f^{(n+1)}(x) = q f^{(n)}(x) + w \text{LSB}(f^{(n)}(x)) = q^{n+1} x + w \sum_{k=0}^{n-1} q^{n-k} \text{LSB}(f^{(k)}(x)) + w \text{LSB}(f^{(n)}(x)),$$

which is the claimed sum at $n + 1$ (the last term is the $k = n$ summand). $\square$

This is exactly what a chained circuit of n copies of the $qx + w$ arithmetic (Section 3.6 or appendix A) computes: each successive copy contributes one new term $\text{LSB}(f^{(k)}(x))$, weighted by q^{n-1-k} , to the final register, and Proposition 2 is the closed-form statement of that fact, it does not remove the need to compute each $\text{LSB}(f^{(k)}(x))$ in turn (they are genuinely different values at

each k), but it gives an explicit, verifiable target for what an n -step circuit must equal. In the two section A and B of appendix A describe various circuits hence different quantum algorithms for different q and w .

The symmetric case $q = w$

When $w = q$, a scalar-factor identity proved below (Proposition 6, $\text{LSB}(q^m y) = \text{LSB}(y)$ for odd q) collapses most of the sum. Note $f^{(k)}(x)$ is divisible by q for every $k \geq 1$ (an immediate induction from $f^{(k+1)}(x) = q(f^{(k)}(x) + \text{LSB}(f^{(k)}(x)))$), so write $r_k := f^{(k)}(x)/q$ for $k \geq 1$ (an integer) and $\text{LSB}(f^{(k)}(x)) = \text{LSB}(q r_k) = \text{LSB}(r_k)$. Substituting into Proposition 2 with $w = q$:

$$f_{q,q}^{(n)}(x) = q^n(x + \text{LSB}(x)) + \sum_{k=1}^{n-1} q^{n-k} \text{LSB}(r_k), \quad r_k = \frac{f_{q,q}^{(k)}(x)}{q}. \quad (202)$$

The $k = 0$ term absorbs into the leading $q^n(x + \text{LSB}(x))$ exactly because $f^{(0)}(x) = x$ is not itself guaranteed divisible by q , while every later term is expressed via the *reduced* sequence r_k rather than the raw, q -fold-larger iterate $f^{(k)}(x)$; so in the symmetric case the formula is genuinely simpler, not just relabelled. This simplification and using $\text{LSB}(q^m y) = \text{LSB}(y)$ leads to a more optimal circuit design. This is consistent with, and gives the general algebraic reason behind, the odd-core reduction $y_{k+1} = q(y_k + 1)/2^{v_k}$ used in Appendix B for $(q, w) = (7, 7)$: r_k and the odd core y_k of Appendix B are related (both strip a factor tied to q and to LSB respectively) but are not the same sequence, since r_k still carries whatever power of 2 divides $f^{(k)}(x)/q$, while y_k is fully odd.

3.4 The general algebraic map $g_{q,w}$ and its convergence criterion

The direct, bitwise-computable counterpart of Section 3.2's analytic equation: for odd integers $q, w > 0$, define

$$g_{q,w}(x) := qx + w\text{LSB}(x). \quad (203)$$

Proposition 3. *Write $x_0 = 2^{S_0}u_0$ with u_0 odd. Then for every $n \geq 0$, $x_n = 2^{S_n}u_n$ with u_n odd, where u_{n+1} is obtained from u_n by stripping all factors of 2 from $qu_n + w$ (always even, since q, u_n, w are all odd) and $S_{n+1} = S_n + b_n$ (b_n the number of factors stripped. This holds for every odd q, w , with no further condition.*

Proof. qu_n is odd (odd times odd), and w is odd, so $qu_n + w$ is even; write it as $2^{b_n}u_{n+1}$ with u_{n+1} odd. Then $x_{n+1} = qx_n + w\text{LSB}(x_n) = q2^{S_n}u_n + w2^{S_n} = 2^{S_n}(qu_n + w) = 2^{S_n+b_n}u_{n+1}$, completing the induction. $\square$

Verified against over 1000 random (x_0, q, w, n) instances with q, w ranging over several odd values, no discrepancy.

This much is unconditional. What is *not* unconditional is whether the odd-core sequence u_n eventually settles at $u = 1$, the one case that makes x_n an exact power of two forever after.

Proposition 4. *$u^* = 1$ is a fixed point of the odd-core map ($u_n = 1 \Rightarrow u_{n+1} = 1$) if and only if $q + w$ is itself a power of two.*

Proof. $u = 1$ gives $qu + w = q + w$; stripping powers of 2 from $q + w$ returns exactly 1 again when $q + w$ is itself a power of 2, and returns something > 1 otherwise. $\square$

This is not a property that holds for every odd q once $w = 1$ is fixed, and the failure is not a corner case. For example the reader can verify that $q = 5, w = 1$ ($q + w = 6$, not a power of 2), starting the odd-core map at $u = 1$ gives $5(1) + 1 = 6 = 2 \times 3 \rightarrow u = 3$, then $5(3) + 1 = 16 = 2^4 \times 1 \rightarrow u = 1$: an exact 2-cycle $\{1, 3\}$, not a fixed point, confirmed by direct computation. So for $q = 5$ the trajectory oscillates between an exact power of two and $3 \times$ (a power of two) forever; it never becomes and stays a pure power of two. For several examples the reader can verify; odd q with $w = 1$ against Proposition 4's criterion directly: $q = 3$ ($q + 1 = 4$, holds), $q = 7$ ($q + 1 = 8$, holds), $q = 15$ ($q + 1 = 16$, holds); $q = 5$ and $q = 9$ (neither $q + 1$ a power of 2) fail, and indeed no fixed point exists for either. So the criterion holds exactly for the Mersenne-adjacent family $q = 2^b - 1$ with $w = 1$, of which $q = 3$ is the smallest, and the classical case, and fails otherwise.

For (q, w) outside this family, $u^* = 1$ need not be reached, and even when some $u^* > 1$ is a fixed point instead (as in Appendix B's (7, 7) system, the reader can readily check that where $q + w = 14$ is not a power of 2 but $u^* = 7$ is fixed: $7(7) + 7 = 56 = 2^3 \times 7$), x_n can never become a pure power of two at all once $q \mid x_n$ for $n \geq 1$ (true whenever $q = w$; see Appendix B). The correct general statement is a dichotomy, the same shape as the Collatz conjecture itself: *either* the odd-core sequence u_n eventually enters some cycle (of any length, at any value, not necessarily $u = 1$), *or* it grows without bound forever. A clean, checkable target (a single bitwise test, as used in Section 5's oracle) is available only in the first case, and only cleanly when that cycle is the fixed point $u^* = 1$; i.e. only for (q, w) with $q + w$ a power of two. Outside that family, verifying the dichotomy for a specific x_0 falls back to actual cycle detection (Section 5's outcome 3, Appendix B's approach), with no bitwise shortcut available.

3.5 The classical special case $q = 3, w = 1$

Specialising Proposition 4: $q + w = 4 = 2^2$, a power of two, so $u^* = 1$ is a fixed point; this is the algebraic reason the classical Collatz map admits a clean “eventually a pure power of two” criterion at all, and it is a comparatively rare property among the $qx + w$ family, not a generic one.

Definition 3. Write $g := g_{3,1}$, i.e.

$$g(x) := 3x + \text{LSB}(x), \tag{205}$$

the specialisation of Section 3.4's general map to $q = 3, w = 1$. This is the map whose behaviour Proposition 5 below ties to f , the classical Collatz function of Section 1 ($f(x) = x/2$ for even x , $3x + 1$ for odd x): g is not a new object introduced for its own sake, it is the one bitwise, branch-free reformulation of f that this whole construction is derived from..

Proposition 5. For g as just defined, $x_n = g^{(n)}(x_0)$ is eventually and permanently an exact power of two if and only if the standard Collatz conjecture holds for x_0 under f .

Proof. Immediate from Propositions 3 and 4 specialised to $q = 3, w = 1$: since $q + w = 4$ is a power of two, $u^* = 1$ is a genuine fixed point of the odd-core map, and by Proposition 3 the odd-core map is exactly the standard accelerated Collatz map ($u \mapsto (3u + 1)/2^{\nu(3u+1)}$) – the same map f induces on odd numbers once even steps are contracted away. So x_n eventually and permanently equals 2^{S_n} exactly when this accelerated map reaches $u = 1$, which is exactly the classical Collatz conjecture for x_0 under f ; and once $u_n = 1$, Proposition 4 guarantees it stays 1 forever after, so the power-of-two property persists. $\square$

We checked this computationally against the direct recurrence for $x_0 < 2000$ (all reach an exact power of two, consistent with CC being

classically verified in that range) and against the structural decomposition $x_n = 2^{S_n}u_n$ term-by-term for several x_0 up to 10^6 ; both match exactly. Note the sequence still increases without bound throughout ($3x + \text{LSB}(x) > x$ always), which is expected and not a defect: the claim was never that g mimics f 's up-and-down trajectory step by step, only that it reaches the power-of-two set exactly when f converges.

This is the one case in the entire $qx + w$ family with a single-bitwise- test oracle available at all, which is why the quantum construction of Section 5 targets it specifically.

Remark 3. *One iteration of g is realised reversibly as: (i) a copy of the input register, (ii) the “multiply by 3” adder used in Section 3.6, (iii) the isolate-lowest-set-bit circuit computing $\text{LSB}(x)$ on the copy, (iv) a final ripple-carry addition of the two results. All four stages are standard reversible arithmetic with no division and no conditional branch on parity. Call this circuit U_g ; ω iterations chain ω copies of U_g .*

3.6 An explicit bitwise circuit for $\text{LSB}(x)$

Stage (iii) above is not just asserted to exist; it has a short closed form, valid regardless of which (q, w) is in use, since $\text{LSB}(x)$ itself does not depend on q, w . For an n -bit register ξ and any $p \geq n$, define the prefix-OR

$$\Pi(\xi) = \xi \vee (2\xi) \vee (4\xi) \vee \dots \vee (2^p\xi), \quad (206)$$

where each left shift $2^j\xi$ is a wire relabelling (zero gate cost) and $\vee$ is bitwise OR. Then

$$\text{LSB}(\xi) = \left(2\Pi(\xi) \oplus (2^{2p+1} - 1)\right) \wedge \Pi(\xi). \quad (207)$$

We verified this identity by direct computation against ξ & $(-\xi)$ over 5000 random ξ with p ranging up to 22 bits, with no discrepancies. An equivalent alternative masks the prefix-OR at each shift rather than globally at the end: define

$$K(x, \ell) = x \vee ((2^\ell - 1) \& (x \ll 1)) \vee ((2^\ell - 1) \& (x \ll 2)) \vee \dots \vee ((2^\ell - 1) \& (x \ll \ell)), \quad (208)$$

giving

$$x_{n+1} = 3x_n + ((2^\ell - 1) \& (K(x_n, \ell) \ll 1)) \oplus K(x_n, \ell), \quad (209)$$

or more generally

$$x_{n+1} = qx_n + w((2^\ell - 1) \& (K(x_n, \ell) \ll 1)) \oplus K(x_n, \ell), \quad (210)$$

which we verified directly reduces to $3x_n + \text{LSB}(x_n)$ for ℓ (a bit-width parameter, unrelated to the LSB function) taken larger than the bit-length of x_n ; the two formulas differ only in whether the OR-cascade is globally masked at the end (Π -form) or masked at each shift (K -form).

The circuit built from either form uses three gate primitives only: Toffoli gates (CCX) for the bitwise AND/OR of the prefix-OR tree, CNOT gates (CX) for the XOR and the bit inversions, and a quantum ripple-carry adder for the final $qx_n + w\text{LSB}(x_n)$ combination (with $3x_n$, in the classical case, formed from a shift-and-add, $2x_n + x_n$; general odd q is handled the same way, decomposed into shifted copies of x_n). This gives total gate complexity $O(n)$ per iteration, with no division circuit anywhere in the construction, for any odd q, w .

Remark 4 (This all uses Bennett’s theorem). *Nothing above is a quantum circuit in isolation; each piece (the prefix-OR tree, the AND/OR/XOR combination, the ripple-carry adder) is first a classical Boolean circuit, and what licenses turning it into a unitary, reversible quantum circuit at all is Bennett’s theorem [2]: any classical computation, however implemented, can be simulated by a logically reversible one at the cost of extra “garbage” bits recording enough intermediate state to make every step invertible, and that garbage can then be cleaned up by copying out the desired output and running the reversible computation backwards. Toffoli and CNOT gates are already reversible and are exactly the standard universal gate set for implementing an arbitrary classical Boolean circuit this way (Toffoli alone is universal for classical computation); the “uncompute to erase garbage” step used explicitly in the oracle of Section 5.2 is a direct instance of Bennett’s compute–copy–uncompute construction, not a separate construction. We use this throughout without further comment, but it is worth naming once, since it is the reason any of these classical arithmetic circuits (the adders, the prefix-OR tree, the power-of-two test) are legitimately quantum circuits at all, rather than merely classical circuits described in quantum notation. This remark also applies to the circuits in Appendix A.*

3.7 A scalar-factor reduction lemma

Proposition 6. *For any odd q , any $m \geq 0$, and any positive integer y , $\text{LSB}(q^m y) = \text{LSB}(y)$.*

Proof. q^m is odd, so it contributes no factor of 2; the 2-adic valuation of $q^m y$ equals that of y , hence so does LSB . $\square$

This lets common scalar factors of q be cancelled symbolically rather than computed: for instance $\text{LSB}(3x + 3\text{LSB}(x)) = \text{LSB}(3(x + \text{LSB}(x))) = \text{LSB}(x + \text{LSB}(x))$ (the $q = 3$ case), verified directly for a range of x . When evaluating $\text{LSB}(g^{(k)}(x))$ as part of a multi-step jump operator, this means scalar powers of q that would otherwise require expanding and re-multiplying the whole register at each level can instead be tracked symbolically and cancelled, reducing the gate depth of a k -step composed jump operator from the naive $O(k^2)$ (each level re-expands the full polynomial in x) to $O(k)$ (each level adds a bounded amount of new circuitry). We state this as a design principle rather than a fully worked circuit count here; the saving follows directly from Proposition 6 but a tight accounting of ancilla and depth per level is left for a full circuit-synthesis treatment. This is the identity already used in Section 3.3’s symmetric-case simplification above.

4 Character-Sum Representations of $\text{LSB}(x)$, and Their Fit With Quantum Phase Estimation

Section 3.2 built one analytic representation of $\text{LSB}(x)$ from a concentration argument (a limit as $p \rightarrow \infty$). There is a second, more classical route to the same kind of object, via *finite character sums* rather than a limiting process, and it connects more directly to standard number-theoretic machinery and to a specific quantum primitive (phase estimation) worth naming precisely.

4.1 An exact, finite character-sum formula for $\nu_2(x)$

Proposition 7. *For every positive integer x ,*

$$\nu_2(x) = \sum_{k=1}^{\infty} \frac{1}{2^k} \sum_{j=1}^{2^k} e^{2\pi i j x / 2^k}, \quad (211)$$

where the sum is finite in the strong sense that every term with $k > \nu_2(x)$ is exactly zero, not merely small.

Proof. By the standard orthogonality relation for additive characters of $\mathbb{Z}/n\mathbb{Z}$, $\frac{1}{n} \sum_{j=0}^{n-1} e^{2\pi i j x/n}$ equals 1 if $n \mid x$ and 0 otherwise (the sum is a geometric series in $e^{2\pi i x/n}$, which is 1 exactly when $n \mid x$, and telescopes to 0 otherwise); the sum from $j = 1$ to n is the same set of terms cyclically relabelled. Taking $n = 2^k$, the k -th summand is exactly the indicator of $2^k \mid x$, which is 1 for $k = 1, \dots, \nu_2(x)$ and exactly 0 for all $k > \nu_2(x)$. The total is therefore exactly $\nu_2(x)$, attained already at the finite truncation $k = \nu_2(x)$. $\square$

Can also be verified with direct computation of $\nu_2(x)$ for a range of x in $\nu_2(x)$, confirming both the exact value and the exact truncation point. Consequently $\text{LSB}(x) = 2^{\nu_2(x)}$ is an exact, finite exponential-sum expression with no limit required – a different starting point than equation (6.7)’s $p \rightarrow \infty$ concentration argument, though the two are representing the same underlying quantity from different angles (one via exact orthogonality, one via a decay-rate argument).

4.2 A multiplicative decoupling identity

Proposition 8. *Write $x = \text{LSB}(x)y$ with $y = x/\text{LSB}(x)$. Then for any q, w ,*

$$\text{LSB}(qx + w\text{LSB}(x)) = \text{LSB}(x) \cdot \text{LSB}(qy + w). \quad (212)$$

Proof. $qx + w\text{LSB}(x) = \text{LSB}(x)(qy + w)$ directly. Since $\text{LSB}(2^S m) = 2^S \text{LSB}(m)$ for any integer m (multiplying by a power of 2 shifts the 2-adic valuation up by exactly that power, regardless of m ’s own parity), taking $2^S = \text{LSB}(x)$ gives the claim. $\square$

Verified directly against 3000 random (x, q, w) triples with no discrepancy. This resolves, in the negative, a natural question a reader may ask: whether $\text{LSB}(qx + w\text{LSB}(x))$ could be written as a *finite linear combination of $\text{LSB}(x)$ with constant coefficients*, $\sum_i b_i \text{LSB}(x)$. Any such combination is just a fixed multiple of $\text{LSB}(x)$, but Proposition 8 shows the true multiplier is $\text{LSB}(qy + w)$, which ranges over every power of 2 as y varies ; not a constant, and not a finite set of constants. So no such finite linear form can hold in general, and Proposition 8 is the correct answer to the question above: the relationship is multiplicative, not linear, and the multiplier is exactly as unpredictable as LSB itself. We did not investigate linear type approximations to above equation.

Remark 5 ("Conservation of difficulty" under analytic continuation). *Combining Propositions 7 and 8: if $\text{LSB}(qy + w)$ is expanded via the character-sum (or the equation (6.7)) representation, the multiplicative identity becomes a statement about constructive/destructive interference of oscillating exponentials in the phase $qy_k + w$ at each step k . This probably does not make the dynamics easier to predict ; it relocates the question “what is ν_2 of the next odd core” into “do these phases align”, which is the same question expressed in a different branch of mathematics. We record this because conclusion because this new reformulation may allow it to be analysed by different methods.*

4.3 Relation to classical exponential-sum methods

Proposition 7’s building block, $\frac{1}{n} \sum_j e^{2\pi i j x/n}$, is the finite Fourier / Ramanujan-sum apparatus used throughout analytic number theory to extract exact divisibility and congruence information from arithmetic functions (Ramanujan’s original 1918 use of $c_q(n) = \sum_{\gcd(a,q)=1} e^{2\pi i a n/q}$ to expand arithmetic functions is the classical antecedent). This is the same tool, exponential sums over roots of unity used to detect an arithmetic condition , that underlies the Hardy–Littlewood circle

method, though the resemblance should be stated precisely rather than left vague: the circle method integrates $e(\alpha n) = e^{2\pi i \alpha n}$ over a *continuum* $\alpha \in [0, 1)$ and extracts an asymptotic count by separating major arcs (near low-denominator rationals) from minor arcs, typically for additive representation problems (Waring's problem, Goldbach-type questions). What we have here is the finite-group special case restricted to denominators 2^k , and it extracts an *exact* value (a divisibility indicator), not an asymptotic density, because we are detecting an exact algebraic condition rather than counting solutions to an additive equation. So the connection is they both live in the family of exponential-sum techniques, and a reader familiar with the circle method will recognise the apparatus immediately ; but it is a shared toolkit, not a shared theorem; we are not invoking, and do not need, the major/minor arc machinery itself.

4.4 Why this is a natural fit for phase-estimation circuits

The reason this representation is interesting for the quantum construction specifically, beyond the classical framing above: the kernel $e^{2\pi i j x / 2^k}$ is exactly the phase kernel that Quantum Phase Estimation (QPE) is built to extract. QPE prepares a control register in superposition, applies controlled- U^{2^j} operations for a unitary U with eigenphase ϕ , and reads off ϕ (to n bits) via an inverse QFT; the semiclassical variant used in Shor's algorithm extracts ϕ one bit at a time via a sequence of controlled operations interleaved with single-qubit measurement and classical feedback. Proposition 7 says $\nu_2(x)$ is built from exactly this family of phase kernels, summed one modulus 2^k at a time – structurally the same shape as extracting a phase bit by bit. This gives a precise reason to expect a phase-estimation-style circuit for $\text{LSB}(x)$ to exist, sharper than the informal appeal to phase gates in Section 3.2's earlier remark. It is not, however, a claim that such a circuit is built here, or that it would improve on the exact bitwise circuit of Section 3.6: phase estimation circuits typically carry their own overhead (an inverse QFT, and either extra ancilla or a measurement-and-classical-feedback loop), and the bitwise circuit already achieves $O(n)$ gates with a fully unitary, feedback-free construction. Whether a phase-estimation-based alternative offers any advantage , in gate count, in fitting more naturally into a larger phase-estimation-based algorithm elsewhere in the pipeline, or in neither , is an open question we have not investigated, and we flag it as such rather than assume the structural resemblance implies a practical gain.

5 The Quantum Algorithm

This section targets the classical case $q = 3, w = 1$ of Section 3.5 specifically, since it is the one case in the $qx + w$ family with a clean, single-bitwise-test oracle available (Proposition 4). Section 6 discusses what changes for other (q, w) . Note in appendix C we give "a sliding window version of this quantum algorithm" which is a potentially faster quantum algorithm.

5.1 The algorithm, stated step by step

The algorithm is given here as a sequence of steps, and it is worth giving this one as exactly that before dissecting each piece in the subsections that follow. Fix the search domain $D = \{1, 3, 5, \dots, 2^{\alpha+1} + 1\}$ (odd starting values only; even values are redundant, since two steps of the classical map take any even x to a smaller value already covered by an odd starting point), $N = |D|$, a step bound $t = \lfloor \eta_0 \log x \rfloor$, and the oracle $O(t, x)$ of Section 5.2 below. One run of the algorithm is:

- **Step 1 (Prepare the uniform superposition).** Initialise an $n = \lceil \log_2 N \rceil$ -qubit index

register to $|0\rangle^{\otimes n}$ and apply a Hadamard gate to each qubit:

$$|\psi_0\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x \in D} |x\rangle. \quad (213)$$

(A further ancilla qubit prepared in $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ is used for phase kick-back when the oracle is implemented as a classical predicate $O(t, x) \in \{0, 1\}$ flipping the ancilla's sign; we suppress it below and write the oracle's action directly as a phase flip on the index register, as is standard.)

- **Step 2 (Identify the good/bad decomposition).** Let $k = |\{x \in D : O(t, x) = 1\}|$ be the (a priori unknown) number of “good” states ; candidates not yet showing convergence within t steps, i.e. potential counterexamples to CC or CC' on this run. Writing

$$|\text{good}\rangle = \frac{1}{\sqrt{k}} \sum_{x: O(t, x)=1} |x\rangle, \quad |\text{bad}\rangle = \frac{1}{\sqrt{N-k}} \sum_{x: O(t, x)=0} |x\rangle, \quad (214)$$

the uniform state of Step 1 decomposes as

$$|\psi_0\rangle = \sin \theta |\text{good}\rangle + \cos \theta |\text{bad}\rangle, \quad \sin \theta = \sqrt{k/N}. \quad (215)$$

Note we do not assume $k = 1$ and skipped to the iteration count formula below. We do not: k is unknown, and on a range where CC has already been checked exhaustively the prior is $k = 0$, not $k = 1$, which is exactly why Step 3 exists.

- **Step 3 (Estimate k).** Run the quantum-counting (amplitude-estimation) procedure of Section 5.3 on O to obtain $\hat{k}$, an estimate of k with a confidence interval $[k_{\min}, k_{\max}]$. If $k_{\max} < 1$, stop here and report outcome 1 of Section 5.4 (no violation found on this run). If $k_{\min} > 0$, existence of a counterexample on d is already established at this point, by counting alone – Step 4 below is needed only to exhibit its concrete value, not to prove its existence (Section 5.3 discusses this distinction in full).
- **Step 4 (Grover iterations).** Apply the Grover iteration operator $G = (2|\psi_0\rangle\langle\psi_0| - I) O_\chi$ (a reflection about $|\psi_0\rangle$ composed with the oracle's phase flip O_χ) repeatedly:

$$G|\phi_j\rangle \rightarrow |\phi_{j+1}\rangle, \quad j = 0, 1, \dots, H-1, \quad |\phi_0\rangle = |\psi_0\rangle, \quad (216)$$

for

$$H = \left\lceil \frac{\pi}{4} \sqrt{\frac{N}{\hat{k}}} - \frac{1}{2} \right\rceil \quad (217)$$

iterations (round to the nearest integer) ; the standard optimal count for amplitude amplification with an estimated $\hat{k}$ marked items out of N , using the estimate from Step 3 in place of an assumed $M = 1$.

- **Step 5 (Measure and classify).** Measure the index register, giving a candidate $x^* \in D$. Repeat Steps 1–5 for $T \geq 30$ independent runs and take the majority-occurring outcome (suppressing Grover/estimation failure probability, per Section 5.4). Run the classical post-processing of Section 5.4 on the extracted x^* (the power-of-two bitwise check, and if needed the cycle-detection procedure of outcome 3) to determine which of the four outcomes applies.

The remaining subsections of this section give the oracle (Step 1's $O(t, x)$, used throughout), the counting procedure (Step 3 in detail), and the classification (Step 5 in detail) each in full.

5.2 Oracle

Fix an interval of starting values $d = \{1, 2, \dots, 2^\Delta\}$ and a candidate step bound $t = \lfloor \eta_0 \log x \rfloor$. By Proposition 5, testing convergence of x under f is exactly testing whether $g^{(t)}(x)$ is an exact power of two, which is a single bitwise check (y is a power of two iff $y \& (y - 1) = 0$). Define

$$O(t, x) = \begin{cases} 1 & g^{(t)}(x) \& (g^{(t)}(x) - 1) \neq 0 \\ 0 & \text{otherwise.} \end{cases} \quad (218)$$

so $O(t, x) = 1$ marks x as *not yet* showing the expected behaviour within t steps. O is computed by t applications of U_g followed by this bitwise test, uncomputed afterward to erase garbage (Bennett’s compute-copy-uncompute construction, Section 3.6), and it costs $O(t) = O(\log x)$ arithmetic-circuit layers per query, this is the source of the quantum speed-up, and it holds regardless of whether CC' is true, since η_0 is only a candidate value being tested. Unlike a test against the fixed set $\{1, 2, 4\}$, this oracle needs no case-by-case target update as t grows: by the Proposition, once the power-of-two condition is met once it holds at every later step too, so a single bitwise test at time t is conclusive for “has converged by step t ”, with no risk of a false positive at an earlier, coincidental power of two. As a phase oracle acting on $|w\rangle$ over the odd search domain $D = \{1, 3, 5, \dots, 2^{\alpha+1} + 1\}$, one round of the search algorithm is the standard Grover layout, with the arithmetic of Section 3.6 supplying the middle stage:

$$|0\rangle \rightarrow -[\mathbf{H}^{\wedge} \mathbf{n}] - [\mathbf{U_g}^{\wedge} \mathbf{t}] - [\mathbf{Phase\ Oracle\ } O(\mathbf{t}, \mathbf{w})] - [\mathbf{Diffusion\ D}] - \rightarrow \text{measurement} \quad (219)$$

i.e. a layer of Hadamards to prepare the uniform superposition over D , t applications of U_g computing the candidate trajectories in superposition, the phase oracle $O(t, w)$ above, and the usual Grover diffusion operator, iterated and wrapped in the amplitude-estimation procedure of Section 5.3.

5.3 Quantum counting before Grover

A weakness of using Grover directly is that its optimal iteration count depends on the number of marked items, which is unknown, and on $[1, 2^{68}]$ the known prior is that this number is 0. We therefore run quantum amplitude estimation (“quantum counting”) [8] on O *before* Grover, as a distinct preprocessing algorithm with its own steps, given in full below.

The counting algorithm, stated step by step

Quantum counting is phase estimation applied to the Grover operator G of Step 4 (Section 5.1), exploiting the fact that G acts as a rotation by angle 2θ on the 2-dimensional subspace spanned by $|\text{good}\rangle, |\text{bad}\rangle$, where $\sin \theta = \sqrt{k/N}$ as in Step 2. Fix a precision parameter $P = 2^t$ (an integer number of control qubits t).

- **Step A (Prepare control and index registers).** Initialise a t -qubit control register to the uniform superposition and the index register to $|\psi_0\rangle$ of Step 1:

$$\frac{1}{\sqrt{P}} \sum_{j=0}^{P-1} |j\rangle \otimes |\psi_0\rangle. \quad (220)$$

- **Step B (Controlled powers of G).** Apply controlled- G^{2^l} from the l -th control qubit onto the index register, for $l = 0, \dots, t-1$ (the standard phase-estimation primitive), giving

$$\frac{1}{\sqrt{P}} \sum_{j=0}^{P-1} |j\rangle \otimes G^j |\psi_0\rangle. \quad (221)$$

Since G has eigenvalues $e^{\pm 2i\theta}$ on this subspace and $|\psi_0\rangle$ is an equal-weight combination of both eigenvectors, this entangles the control register's phase with $\pm\theta$.

- **Step C (Inverse Quantum Fourier Transform).** Apply the inverse QFT to the t -qubit control register:

$$\text{QFT}^\dagger : |j\rangle \mapsto \frac{1}{\sqrt{P}} \sum_{y=0}^{P-1} e^{-2\pi i j y / P} |y\rangle, \quad (222)$$

which concentrates the control register's amplitude near $y \approx P\theta/\pi$ (or $P(1 - \theta/\pi)$ for the conjugate eigenvector component, resolved using $\sin \theta = \sin(\pi - \theta)$).

- **Step D (Measure and convert to a count).** Measure the control register, obtaining an integer $y \in \{0, \dots, P-1\}$. Set $\tilde{\theta} = \pi y / P$ (folding into $[0, \pi/2]$ as needed) and output

$$\hat{k} = N \sin^2 \tilde{\theta}. \quad (223)$$

- **Step E (Precision guarantee).** By the Brassard-Hoyer-Tapp approximate counting theorem [8], for $P \geq 4$ this procedure uses $P-1$ oracle queries and outputs $\hat{k}$ satisfying

$$|\hat{k} - k| \leq \frac{2\pi}{P} \sqrt{k(N-k)} + \frac{\pi^2}{P^2} N \quad (224)$$

with probability at least $8/\pi^2 \approx 0.811$ (boosted to any constant by repeating and taking the majority vote, as in Step 5 of Section 5.1). Taking $P = \Theta(\sqrt{N})$ makes this error $O(\sqrt{N})$ in the worst case ($k \approx N/2$) and $O(1)$ when k is small, which is the regime of interest here: on $[1, 2^{68}]$ the prior(the guessed number) is $k = 0$ or k very small.

Grover is optional for existence; it is only needed to name the counterexample.

Step E's guarantee already answers the yes/no question of whether a counterexample exists in d : if the resulting confidence interval excludes 0 ($k_{\min} > 0$), that is already a proof that $k \geq 1$, i.e. that some $x \in d$ satisfies $O(t, x) = 1$ – established by the counting algorithm alone, with no Grover iteration run at all. What quantum counting cannot do is say *which* x this is: the estimate $\hat{k}$ carries no information about the location of the marked states, only their number. Grover's algorithm (Step 4 of Section 5.1) is the step that turns “a counterexample exists” into “here is the counterexample”, by amplifying the amplitude on the specific marked x until it can be read off by measurement. So the two algorithms answer different questions, *existence* versus *specific counterexample*, and only the second is optional in the sense that a proof of CC's failure on d does not, strictly speaking, require producing the counterexample's numerical value; only a complete disproof *with an explicit witness*, of the kind outcome 3 in Section 5.4 is built to produce, does.

Hypothesis-test framing. It is convenient to phrase this formally as a hypothesis test. Let k be the true number of counterexamples in $d = \{1, \dots, 2^\Delta\}$:

$$H_0 : k = 0 \text{ (CC holds on } d) \quad H_A : k \geq 1 \text{ (some counterexample is in } d). \quad (225)$$

Amplitude estimation returns $\hat{k}$ together with a confidence interval $[k_{\min}, k_{\max}]$ set by the phase-estimation resolution (finer resolution – more rounds – narrows the interval at the cost of more queries). This gives a decision rule with the same two cases as above, stated more precisely:

- **Fail to reject H_0 :** $k_{\max} < 1$, i.e. the interval contains only 0. Report no counterexample found on d (evidence, not proof, as above).
- **Reject H_0 :** $k_{\min} > 0$, i.e. the interval excludes 0. Proceed to Grover’s algorithm, tuned to $\hat{k}$, to extract a counterexample.

When the interval straddles 0 (contains it but $k_{\max} \geq 1$), neither decision is warranted; the estimate is refined with additional amplitude-estimation rounds before proceeding.

5.4 The four outputs

Running the above for a run of $T \geq 30$ repetitions (so that a majority vote suppresses Grover/estimation failure probability) yields one of four classified outcomes:

1. **CC’ true and CC true.** $\hat{k} \approx 0$ for the tested η_0, Δ . Evidence for both CC and CC’ on the tested range, nothing more.
2. **CC’ false and CC true.** A trajectory has not reached an exact power of two within $\lfloor \eta_0 \log x \rfloor$ steps but does so once run further. Response: increase η_0 and rerun; this is not a counterexample to CC.
3. **CC’ false and CC false.** This outcome requires an actual cycle to be exhibited, not merely a failure to converge ; the latter is indistinguishable from divergence and belongs to outcome 4 below. The classical post-processing step is a standard cycle-detection procedure (Floyd’s or Brent’s algorithm) run classically on the extracted x : maintain two pointers iterating f at rates 1 and 2, and check for pointer coincidence; the original formulation of the Collatz conjecture may be used for detecting cycles (without using LSB). If and when they coincide, the cycle is exhibited explicitly and finitely; if that cycle’s elements differ from $\{1, 2, 4\}$, this is a rigorous disproof of CC for x (by Proposition 5, since x ’s g -trajectory then never becomes an exact power of two) and, since $CC' \Rightarrow CC$, of CC’ too.

This step’s running time is not bounded above: if x genuinely diverges rather than cycles, the two pointers never coincide and the search does not terminate, which is exactly why it must be capped by a resource budget (below) rather than run indefinitely. It also is not computationally free even when a cycle does exist: any nontrivial cycle of the classical Collatz map is known to require at least 91 terms and at least 1.375×10^{11} odd elements [?], with more recent computational verification pushing the minimum cycle length further still, to on the order of 2.18×10^{11} elements under the accelerated (odd-only) map, or 3.56×10^{11} without the acceleration [5] (check the current record before submitting; this bound is revised periodically as verification extends). Beyond a numeric floor, the cycle’s *period* is also structurally constrained: Eliahou [6] proved that the period p of any nontrivial cycle must be expressible as

$$p = 301994a + 17087915b + 85137581c, \quad (226)$$

for non-negative integers a, b, c with $b \geq 1$ and $ac = 0$; a consequence of the continued-fraction expansion of $\ln 3 / \ln 2$. This gives cycle detection a second, independent check once a candidate cycle is exhibited: its exact period can be tested against this form, and any period not expressible this way would itself indicate an error in the detection (some sort of error in the classical post-processing, most likely, rather than a disproof of Eliahou’s theorem). So a cycle-detection search cannot possibly terminate before reaching this many iterations if the cycle it is looking for is a classical-style one; whether the same order-of-magnitude lower bound and period constraint apply to cycles of the g -based reformulation used here has not been checked and would need its own argument. So the search is: guaranteed to eventually confirm a cycle *if* one exists and the budget is generous enough to reach it, but not decidable in general within any fixed budget, since exhausting the budget without a coincidence rules out neither a longer cycle nor divergence.

The asymmetry this creates.

1. If a counterexample *is* found within the tested interval, outcome 3 above is a $\Omega(\sqrt{n})$, finite disproof of CC: the algorithm exhibits an explicit integer entering a cycle other than $1 \rightarrow 4 \rightarrow 2 \rightarrow 1$, and it does so having searched the interval quadratically faster (in query count) than the fastest classical search that could find the same counterexample. If no counterexample is found, the algorithm does *not* disprove CC; it only verifies CC on the tested interval (outcome 1), which is a weaker statement than a proof for exactly the reason any bounded search, quantum or classical, cannot certify behaviour outside the range it examined. This asymmetry (finding is conclusive, not-finding is not) is the same one that already governs classical verification of CC, and the quantum algorithm changes the cost of the search, not the logical asymmetry.
2. **Unclassified.** The extracted x neither reaches an exact power of two within a generous re-run of η_0 , nor is a cycle exhibited by the detection procedure of outcome 3 within the allotted budget. This is the “we don’t know yet” case, it absorbs every case where the search simply ran out of resources, which is not the same as a proof of divergence; and requires further classical analysis (e.g. Appendix B’s style of argument, tailored to the specific x). This is the most likely outcome of any single anomaly found in practice, given how extensively CC has already been verified classically.

6 Generalisation to $qx + w$: Circuit Adaptation

The circuit of Sections 3.6–3.7 generalises directly to any odd q : replace the “multiply by 3” block with a “multiply by q ” block built the same way (decompose q into a sum of shifted copies of x , add with the same ripple-carry adder). The oracle, counting/Grover step, and four-output classification of Section 5 apply verbatim to any (q, w) for which Section 3.4’s criterion gives a fixed point at $u^* = 1$ (i.e. $q + w$ a power of two); the bitwise power-of-two test remains the correct, single oracle target. For (q, w) outside that family, the same circuit still computes $g_{q,w}$ correctly, but the oracle target must change to whatever cycle Proposition 3’s dichotomy actually produces for that (q, w) , which in general requires the cycle-detection approach of Section 5’s outcome 3 rather than a single bitwise test; exactly the situation Appendix B investigates for $(7, 7)$.

7 Complexity and Feasibility

Time complexity. Verifying $[1, 2^m]$ classically costs $\Omega(2^m)$ trajectory computations in the worst case. The scheme above costs $O(2^{m/2})$ oracle queries, each of depth $O(m)$ (one U_g layer is $O(m)$ gates via the adder circuits, and $t = O(m)$ layers are chained), so $O(m \cdot 2^{m/2})$ total gate depth against $O(m \cdot 2^m)$ classically ; a quadratic improvement in the exponent, matching the standard Grover speed-up.

Space complexity. The space complexity is polynomial if the stopping time for the trajectories is polynomial; but in the general case the space complexity is proportional to the worst stopping time for some trajectory.

Space complexity does not depend on the unknown true stopping time. It is worth being precise about why the qubit count stays $O(m)$, polynomial, and in fact linear, in the register size, given that the *true* stopping time of a Collatz trajectory (how long it actually takes to reach $\{1, 2, 4\}$, if it ever does) is not known to be bounded by any fixed polynomial in $\log x$, or bounded at all, for every x ; that is essentially what CC and CC' assert. The oracle $O(t, x)$ of Section 5.2 never needs to know or reach the true stopping time: it evaluates $U_g^t(x)$ at a single, *fixed*, chosen cutoff $t = \lfloor \eta_0 \log x \rfloor = O(m)$ and reports whether the power-of-two condition holds *by then* ; a well-defined question with a well-defined answer regardless of what happens after step t , or of whether CC or CC' is true at all. Running t chained applications of U_g needs no more live qubits than a single application: each layer's ancilla is uncomputed (Bennett's construction, Section 3.6's remark) before the next layer begins, so the same $O(m)$ working register is reused t times rather than t copies being held simultaneously. Consequently the qubit count for one full oracle evaluation is $O(m)$, unconditionally, this bound does not improve if CC' happens to be true (trajectories converging by step t) and does not degrade if it is false (trajectories that would need far longer, or forever, to converge): the algorithm simply reports, at fixed cost, what the state looks like at step t , and that snapshot is itself the complete, interpretable output (outcome 1 or 2 of Section 5.4 if it has converged or looks likely to soon; outcome 3 or 4 if not), never a partial result awaiting more space or time to resolve into something else. A trajectory that has not converged by step t is not a computation that ran out of resources, it is already evidence, at the CC' level at minimum, and if it persists as η_0 is increased across repeated runs, evidence of something more unusual, obtained at exactly the same $O(m)$ -qubit, $O(m^2)$.depth cost as a trajectory that converges on the first try.

Two caveats belong in the presentation of this number:

- Squaring the classically verified bound (roughly 2^{68}) to 2^{136} in the same query budget, as sometimes quoted, assumes the quantum circuit runs at the same query rate as the classical verification, which is not established and is unlikely given gate-error and connectivity overheads.
- Hardware. m in the range of interest (60–140) implies registers of that many logical qubits plus the ancilla needed for the adder and comparison circuits, run through $O(m)$ sequential Grover/QAE iterations, each containing $O(m)$ -deep arithmetic. Thus the contribution here is algorithmic, not a claim of present-day feasibility.

8 Conclusion

We presented the results in the order they were discovered: a general analytic, exponential-sum representation of $LSB(x)$ valid for arbitrary odd q, w (Section 3.2); its exact character-sum and closed-form-sum counterparts (Section 3.3, Section 4); the general algebraic map $g_{q,w}$ and the exact dichotomy governing exactly when it admits a clean power-of-two oracle target versus when it requires cycle detection (Section 3.4); and the classical $q = 3, w = 1$ Collatz map as the one specialisation of that theory with a single-bit oracle available at all (Section 3.5). On that classical

case we derived a Grover/quantum-counting-based search algorithm with quadratic query speed-up for verifying CC on a finite range, stated the CC' logarithmic-stopping-time strengthening, and gave the four-way classification of the algorithm's output. We gave the first quantum algorithm that can disprove the CC by potential counterexample.

References

- [1] LOCATE REF
- [2] LOCATE REF
- [3] LOCATE REF
- [4] LOCATE REF
- [5] LOCATE REF
- [6] LOCATE REF
- [7] LOCATE REF
- [8] LOCATE REF
- [9] LOCATE REF

Appendix A

The appendix sketches or describes various classical circuits for the conjecture then gives various sketches or descriptions of quantum circuits.

Section A of Appendix

$LSB'(x)$ is expressed in the language of C++ notation, so as usual $|$, $\&$, $\wedge$ are logic operations on binary digits in C++, and $<<$ is the left shift operation in C++. Define $K(x, L)$ (for clarity) follows

$$K(x, L) = x \mid ((2^L - 1) \& (x << 1)) \mid ((2^L - 1) \& (x << 2)) \mid ((2^L - 1) \& (x << 3)) \dots \mid (((2^L - 1) \& (x << L)))$$
 $K(x, L)$ contains L terms of the form $((2^L - 1) \& (x << s))$ for some natural number $s \in [1, L]$. Then

$$LSB'(x) = ((2^L - 1) \& (K(x, L) << 1)) \wedge K(x, L) \quad (\text{A})$$

which can of course be expressed without $K(x, L)$ -but we use $K(x, L)$ for clarity. L is a suitably large integer and it to be greater than the binary length of the largest integer in the Collatz trajectory of x_0 . Thus the Collatz iteration can be expressed in terms of logical connectives. Which is

$$\begin{aligned}
 x_{n+1} &= 3x_n + ((2^L - 1) \& (K(x_n, L) << 1)) \wedge K(x_n, L) \\
 &= (x_n << 1) + x_n + ((2^L - 1) \& (K(x_n, L) << 1)) \wedge K(x_n, L)
 \end{aligned} \quad (\text{A2})$$

In this form of it can be directly translated into assembly language, because all classical CPUs support *AND*, *OR*, *XOR* on integers as assembly language instructions, with this direct implementation an N -bit CPU can test the Collatz conjecture for the trajectory values starting with x_0 bounded above by the maximum word size of the CPU which is $2^N - 1$. Assembly language is faster than a higher level language with a suitable implementation of the same code. For testing if the trajectory of x_0 converges to the cycle or a power of 2 a CPU can simply to L comparisons for each integer $1, 2, 3, \dots, 2^L$. It can also be implement with arbitrary large integer C++ code regarding the binary digit size of integers.

We sketch a proof of why the Collatz iterate can be computed in quantum polynomial complexity using above. As mentioned above $K(x, L)$ contains L terms of the form $((2^L - 1) \& (x << s))$ for some natural number $s \in [1, L]$ $L = O(\log(x))$ Because it is expressed using bitwise logical operations it means it can be directly translated into a quantum circuit, using the fact that there are quantum analogues of classical logic circuits, therefore the quantum version of equation A can be computed in quantum polynomial time and space complexity By substituting equation A into the system of equation 12 we see that the Collatz iterates takes at most linear $O(x)$ time and space classically in the worst case, hence it shows our quantum algorithm (the oracle) run in linear time in worst case.

The oracle is constructed as follows (a sketched out description-with rough diagrams below-but this has enough detail it can be filled in by

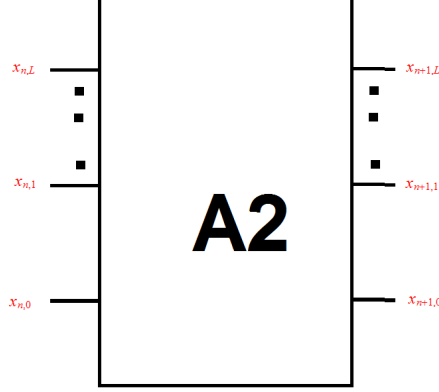


Fig Z1

Thus ω iterations will be (as the red bracket in the diagram below shows)

an expert reader) e.g. the Hadamard gates $H^{\otimes D}$ (for some integer D) are not shown that generate the uniform input superposition. The well known Collatz iteration, zero above, is rewritten as equation 5, which is rewritten as equation A or A2 above. The diagram below shows one iteration of A2 as a quantum circuit which is equivalent to one iteration of the Collatz conjecture where $x_{n,L}$ $x_{n,1}$ $x_{n,0}$ are the qubits that make up the integer value of the iteration x_n and where $x_{n,L}$ is the most significant qubit of x_n and x_0 is the least significant qubit of x_n and where L is the bitlength of the number x_n here.

and it is expected ω is polynomial from above. Then the above implements part of equation 9 i.e. it computes $f^t(2w + 1)$ with $t = \omega$. Therefore to fully implement equation 9 the circuit q which tests if $f^t(2w + 1) \in \{Q\}$ the circuit for the oracle is thus

Of course there may be other ways to achieve the same result using the ideas in this paper to give alternative circuits for the oracle above.

Also we could give a generalised version of our algorithm using the generalised equation for the CC which are equation 6.6 and 6.7.

Note that for the generalised Collatz conjecture expressed by 6.5,6,6 or 6.7 we have the similar to 8.1 the similar observation and equation 8.2

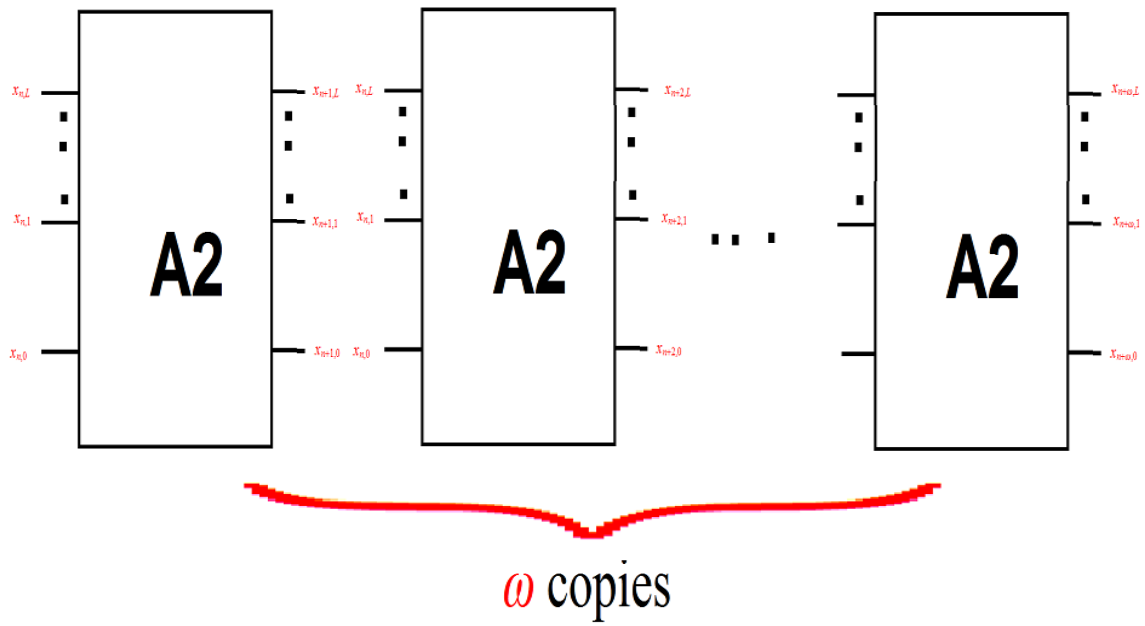


Fig Z3

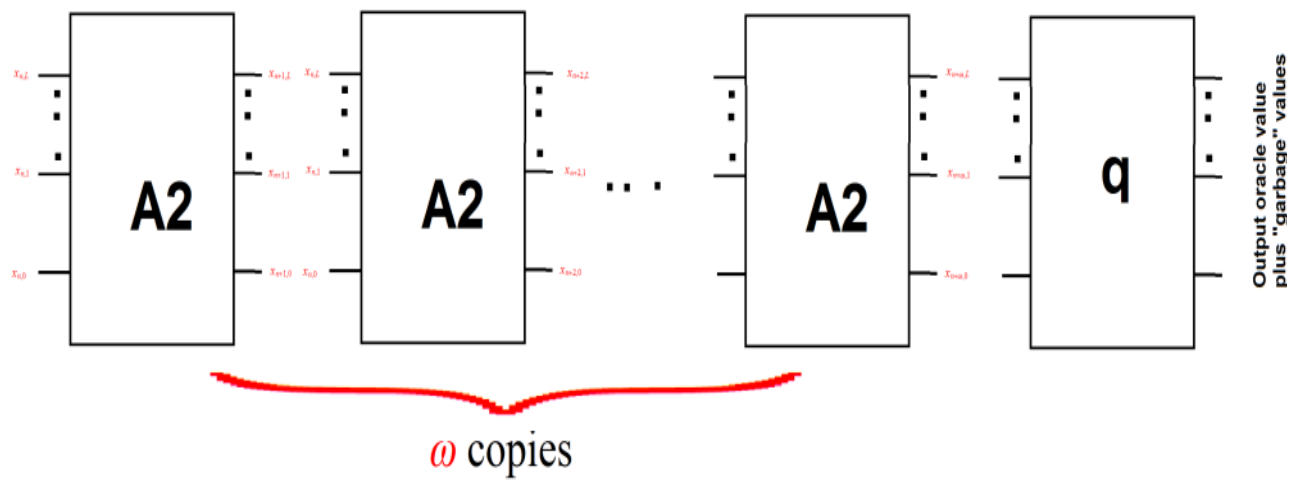


Fig Z4

below As mentioned above. If we can express $L(qx + wL(x))$ in another form e.g.

$$L(qx + wL(x)) = \sum_{\forall i} b_i L(x), b_i \text{ are constants} \quad (8.2)$$

and using above expressions, such as given by 12, then it may lead to a new insight of why some starting values eventually fall into a potential cycle or diverge. The behaviour of $L(qx + wL(x))$ is related to fractal iterations.

Let us consider the circuit in more detail so for the equation

$$x_{n+1} = 3x_n + LSB(x_n) \quad (2)$$

we can use some quantum adder circuit of which there is more than one type (such as a ripple type adder built out of quantum half adder) to compute the $3x_n$ part of equation 1 by adding x_n multiple times to implement multiplication of x_n by some integer. Another way to express $LSB'(\xi)$ where ξ is an p bit number i.e.

$$\xi = \xi_p \xi_{p-1} \xi_{p-2} \dots \xi_1 \quad (A3)$$

where ξ_1 is the least significant bit of ξ and ξ_p is the most significant bit of ξ here. Firstly define

$$\Pi(\xi) = \xi \vee (2\xi) \vee (4\xi) \dots \vee (2^{p-1}\xi) \vee (2^p\xi) \quad (A4)$$

so $\Pi(\xi)$ is the bitwise *OR* operator, $\vee$, applied $p - 1$ times in the above. Then 2^{2p-1}

$$LSB'(\xi) = (2\Pi(\xi) \oplus (2^{2p+1} - 1)) \wedge \Pi(\xi) \quad (A5)$$

* so $LSB'(\xi)$ is computed using a single bitwise *AND* operation ($\wedge$) and a single bitwise *XOR* operation ($\oplus$) as expressed above. Note that evaluating A4 is a polynomial time and polynomial space operation (as it uses the bitwise *OR* operation p times). A5 can also be evaluated in polynomial time and polynomial space. Let us give an example to demonstrate equation A4 and A5. Let $\xi = 20 = 10100_2$ (which has least significant bit 0) and evaluating A4 here we have

•

(A6)

$$\begin{aligned} \Pi(20) &= 10100_2 \vee 101000_2 \vee 1010000_2 \vee 10100000_2 \vee 101000000_2 \vee 1010000000_2 \\ &= 1111111100_2 \end{aligned}$$

and evaluating A5 here gives

$$\begin{aligned}
& (2(1111111100_2) \oplus (1111111111_2)) \wedge 1111111100_2 \quad (\text{A7}) \\
& = ((11111111000_2) \oplus (1111111111_2)) \wedge 1111111100_2 \\
& = ((00000000111_2)) \wedge 1111111100_2 \\
& = ((00000000111_2)) \wedge 01111111100_2 \\
& = 00000000100_2 \\
& = LSB'(20) = 4
\end{aligned}$$

as expected. The above equation A3,A4,A5 combined with known circuits for addition and copying (which does not contradict the no cloning theorem here) are the basis of our novel and new circuits for testing the Collatz conjecture. To describe our novel circuits in more detail we will first give classical version of the circuits (for 3 bit numbers) then give quantum version of the classical circuits for the $3x + 1$ problem. For simplicatiy we do not discuss gate delays to make the explanation clearer though gates delays can be easily added in places if needs be.

The Classical Circuit for $LSB'(\xi)$ For A 3 Bit Numbers ξ

$$\Pi(\xi) = \xi \vee (2\xi) \vee (4\xi) \vee (2^3\xi) \quad (\text{A7})$$

so $\Pi(\xi)$ is the bitwise *OR* operator, $\vee$, applied p times in the above. Then

$$LSB'(\xi) = (2\Pi(\xi) \oplus (2^{6+1} - 1)) \wedge \Pi(\xi) \quad (\text{A8})$$

We write classical logic gates,acting on some binary string , as

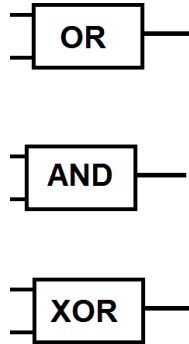


Fig Z6

which is non standard but still obvious what is meant from the labels. A logic gate is understood to act on a certain number of bit (e.g. 8 bit)

on both its inputs and output. So actually, a wire (or cable) in the above three logic gates is implemented as a cable/wire containing 8 sub-wires in a single wire (if it was taking in 8 bit numbers as an input). Variation in the wires input to the logic gates is used to express the circuit more clearly so a OR gate above may be expressed as

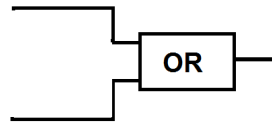


Fig Z7

but has the same meaning as the other or gate. Wires that do not join together but overlap will be shown with a symbol like



or

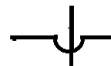


Fig Z8

Wires that do join together in the classical circuit (for copying the signal) are expressed with a



Fig Z9

where the wires join (not that this symbol has a different meaning in the quantum circuit when it is used as part of a CNOT gate), for example



Fig Z10

So a circuit to implement equation A7 is as follows

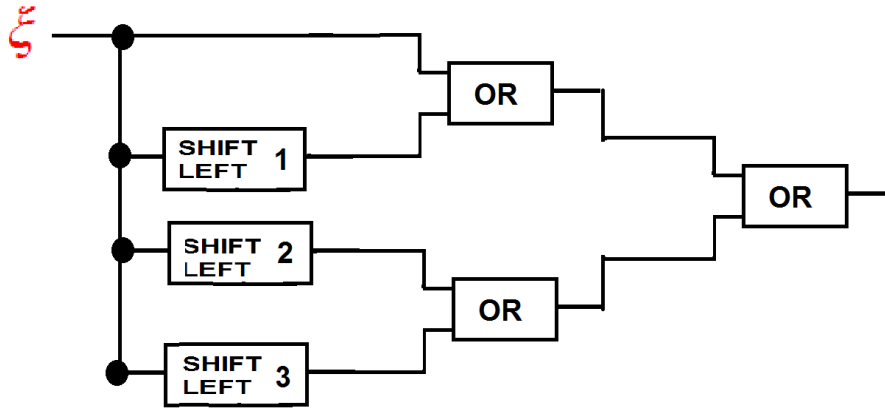


Fig Z11

Then a classical circuit for $LSB'(\xi)$ of a 3 bit number is as follows

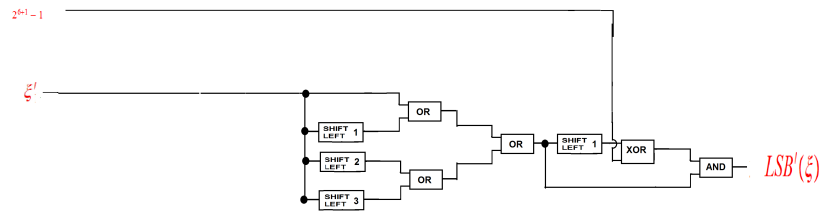


Fig Z12

Thus generalising the above to a p bit number for ξ we have the following circuit for $LSB'(\xi)$

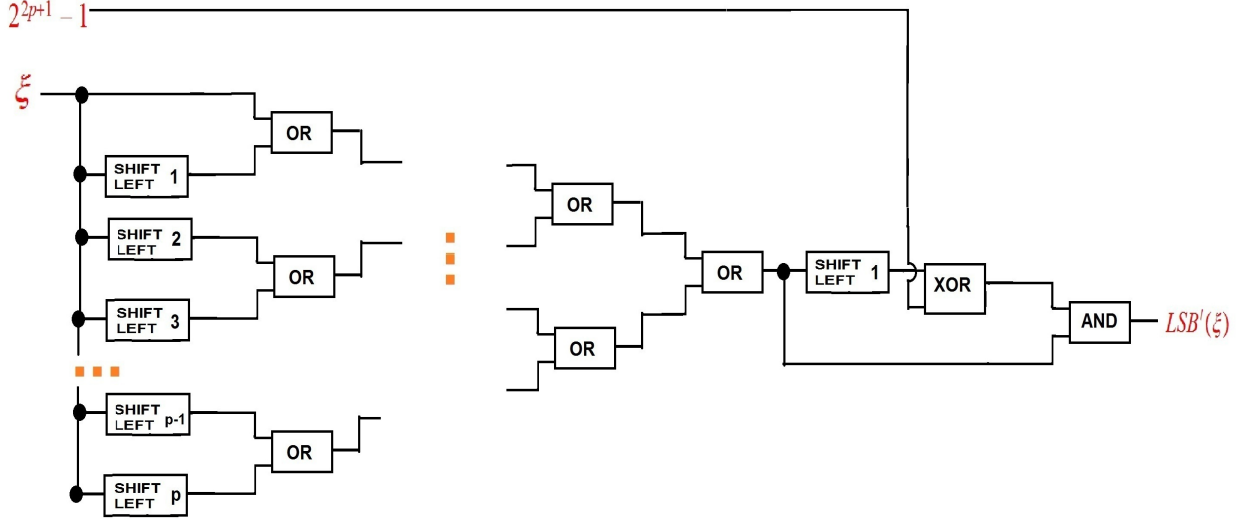
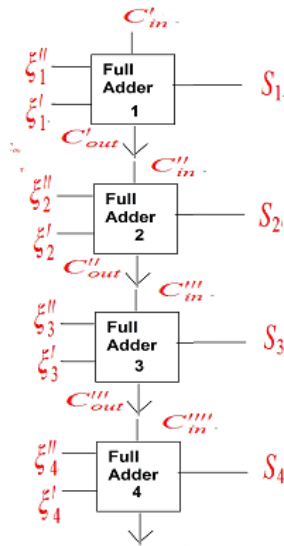


Fig Z13

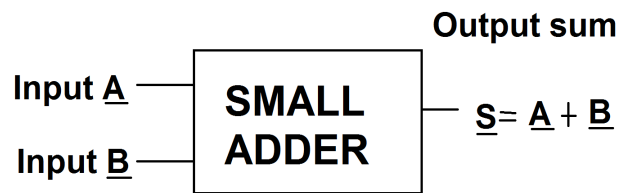
The three orange dots in the above diagram indicate omitted gates but can be filled in as follows. There are p shift left gates where the horizontal three orange dots are, Assume WLOG that p is a power of 2. There is a "pyramid" of OR gates in the area where the vertical three horizontal (square) dots are- by pyramid there are $p - 2$ gates, that are directly connect to the shift left gates indicated near the three horizontal orange (square) dots, these $p - 2$ gates feed into $(p - 2)/2$, these $(p - 2)/2$ gates feed into $(p - 2)/4$ gates and so on until it feeds into the single OR gate (nearest to the XOR gate) as shown in the diagram above. Thus the circuit depth of this "pyramid" of OR gates is polynomial space and polynomial time. Also the value $2^{2p+1} - 1$ is inputted into the generalised circuit above. A classical adder for two 4/3 bit numbers $\xi'' = \xi_4''\xi_3''\xi_2''\xi_1''$ and $\xi' = \xi_4'\xi_3'\xi_2'\xi_1'$ with sum $S = S_4S_3S_2S_1$ and carries $C_{in}'', C_{out}'', C_{in}', C_{out}', C_{in}', C_{out}'$ can be built out of four full adders, as shown in the diagram below, where each full adder is itself built out of a classical half adder.



Flg Z14

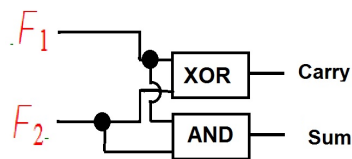
we refer to the above circuit with "small adder" as whea $\underline{A}, \underline{B}, \underline{S}$ are all 4 bit integers in the fig. below

•



Flg Z15

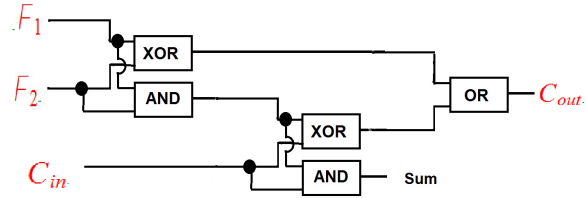
where the circuit for the half adder is on inputs F_1 and F_2 is



Flg Z16

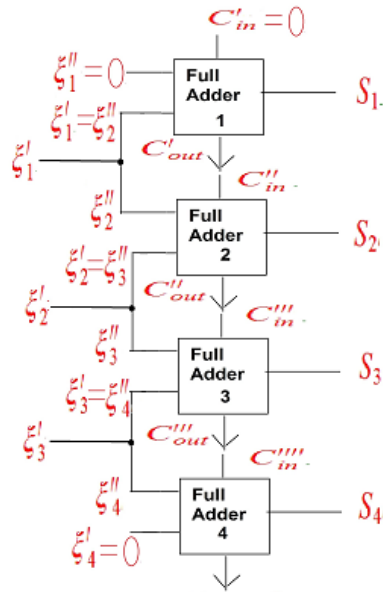
and the circuit for the full adder, on inputs F_1 and F_2 , which can be built from two half adders as follows -where C_{in} is some carry

in and C_{out} is some carry out in the full adder.



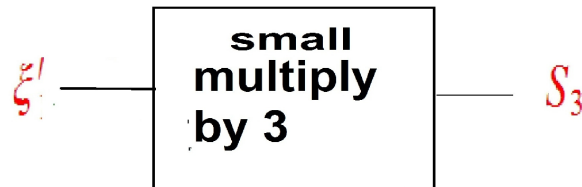
Flg Z17

We can create a multiply by three circuit by letting $\xi'' = \xi' \xi'' = \xi_4'' \xi_3'' \xi_2'' \xi_1''$ and $\xi' = \xi_4' \xi_3' \xi_2' \xi_1'$ and then adding $2\xi''$ and ξ'



Flg Z18

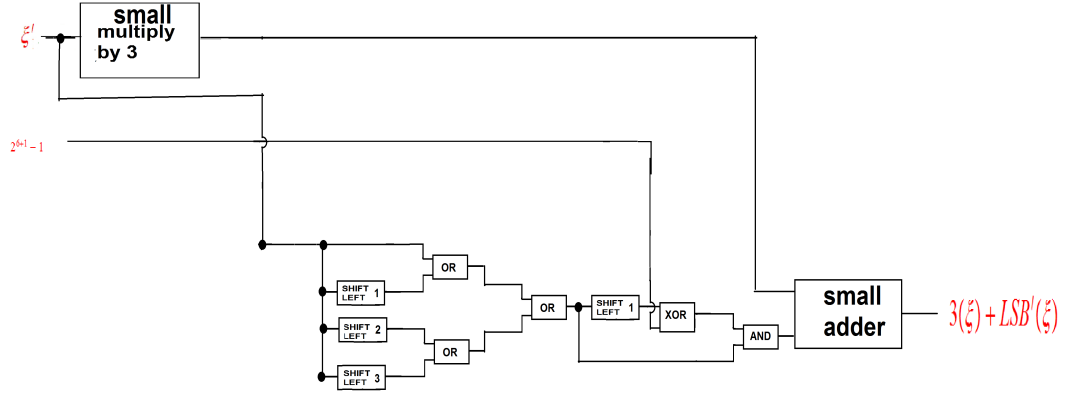
Let represent the above circuit as "small multiply by 3" below (small at its on integers that are 4 bits)



Flg Z20

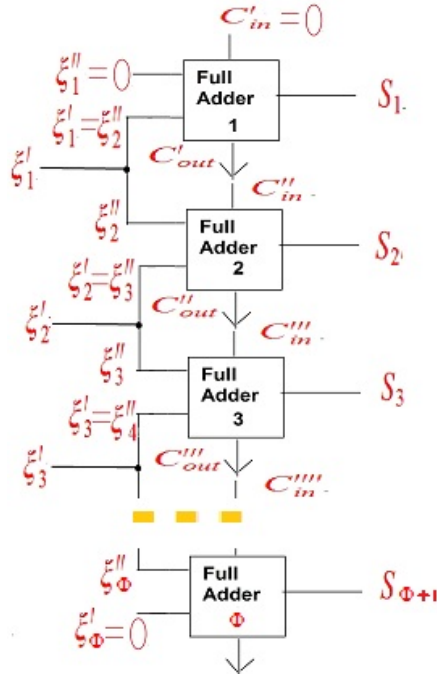
then one iteration for a 4 bit starting value of the Collatz iteration

i.e. is implemented as $3(\xi) + LSB'(\xi)$



Flg Z21

The above is probably the simplest interesting circuit for the Collatz conjecture as it can verify the integers 1, 2, 5, 4 verify the Collatz conjecture. Note that the the above integers are all 4 bits. Now let use generalise the above circuit to any number of bits. We first start by using a strightfoward generalistion of the "small multiply by function| as follows. Let $\xi'' = \xi''_{\Phi} \dots \xi''_4 \xi''_3 \xi''_2 \xi''_1$ and $\xi' = 2\xi''_{\Phi}$ then ξ' has the binary expansion $\xi''_{\Phi} \dots \xi''_4 \xi''_3 \xi''_2 \xi''_1 0$ for some integer Φ which is the binary length of ξ'' .



Flg Z22

where the orange dots above represent the gates in between which are arranged similarly to the gates present in the above fig. We represent the above circuit as "multiply by 3" below (where the integer being multiplied is some arbitrary bit length)

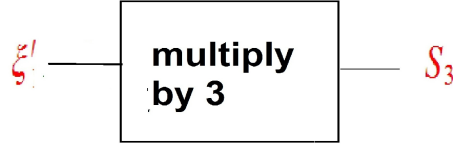


Fig Z23

A straightforward generalisation (to give the known quantum ripple adder circuit) of the above adder to add $\xi'' = \xi''_{\Phi} \dots \xi''_4 \xi''_3 \xi''_2 \xi''_1$ and $\xi' = \xi'_{\Phi} \dots \xi'_4 \xi'_3 \xi'_2 \xi'_1$ as follows

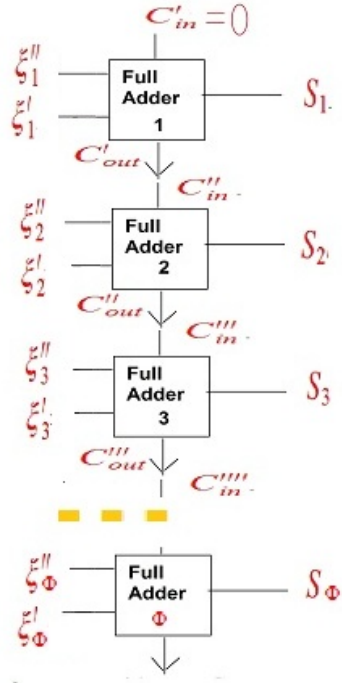
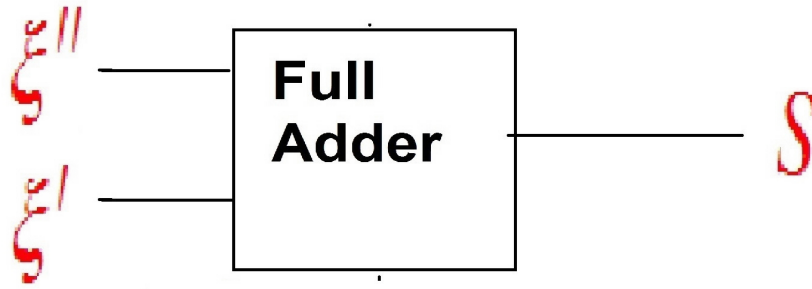


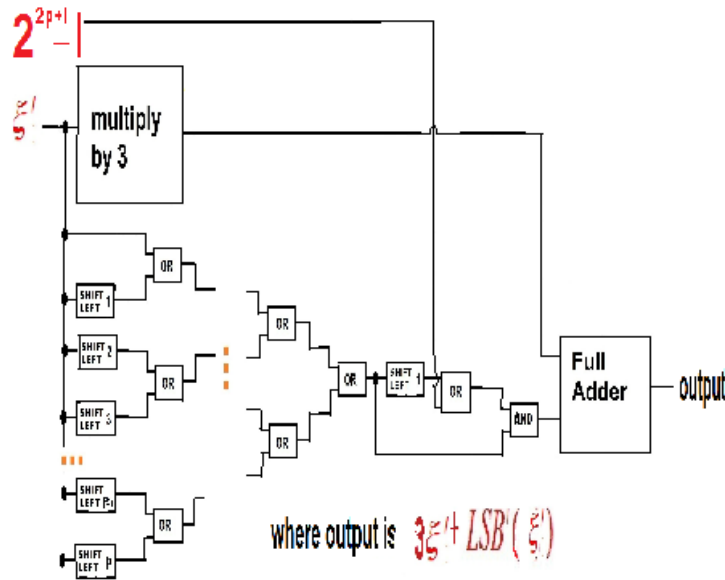
Fig Z24

we represent the above circuit by



Flg Z25

so one iteration of the Collatz conjecture, using our reformulation, for some arbitrary number of any p bit length is expressed by the circuit



Flg Z26

the above circuit can test the Collatz conjecture for all integers that reach the known cycle in one iteration, we refer to the above using the shorthand symbol

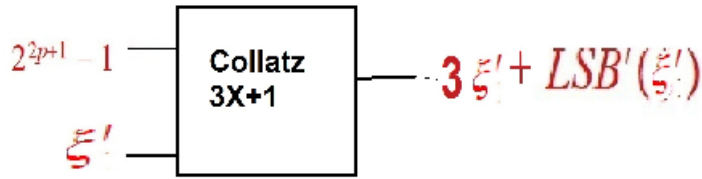


Fig Z27

this a circuit that represents two iterations of the Collatz conjecture is then

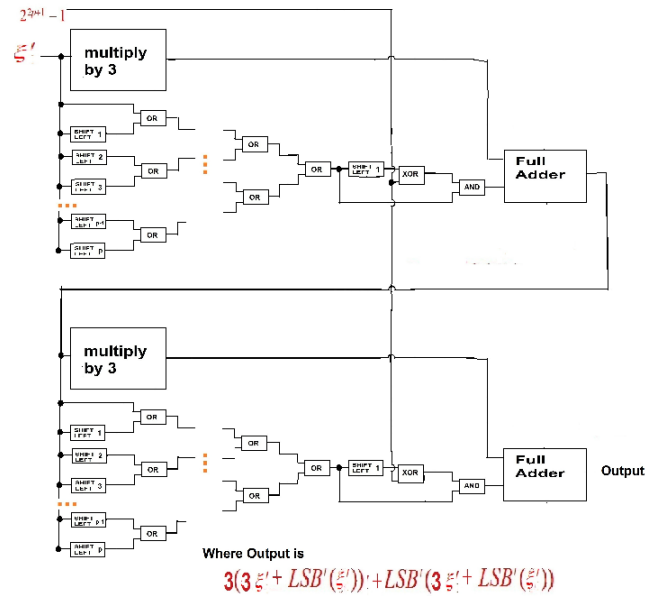
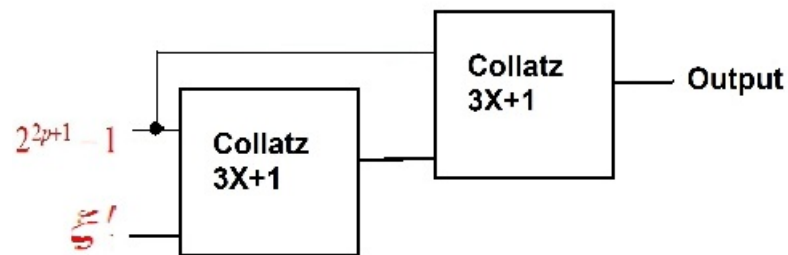


Fig Z28

this can be expressed using the shorthand



Where Output is

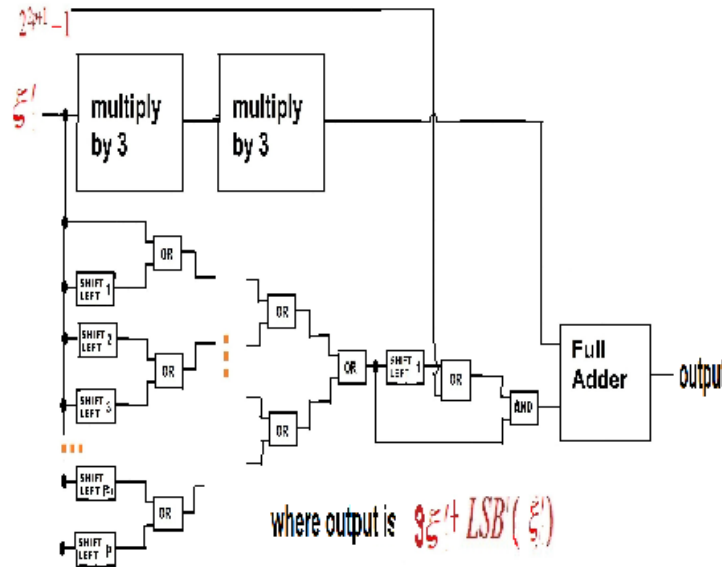
$$3(3\xi' + LSB'(\xi')) + LSB'(3\xi' + LSB'(\xi'))$$

Fig Z29

as mentioned above we would use ω copies of such circuits to test the CC for all integers over a given bounds; a similar remark applies to the $9x + 1$ and $qx + w$ circuits described in the next section . Also mentioned above ω would be expected to be logarithmic in depth.

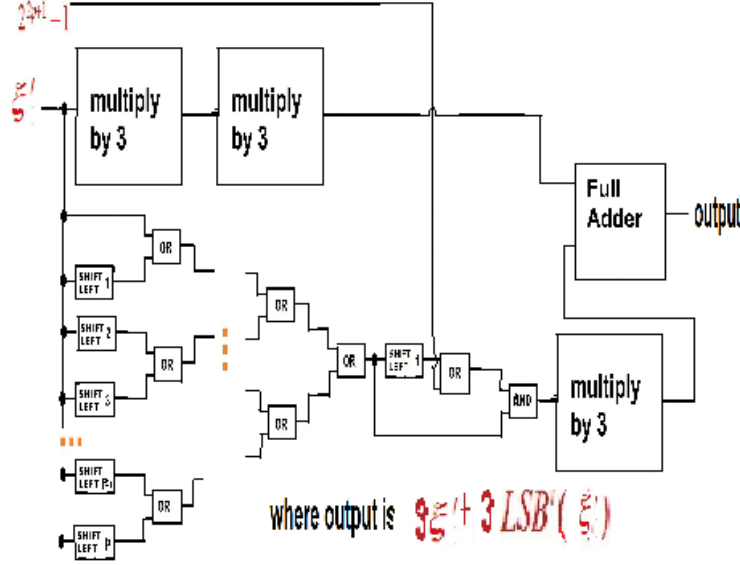
Other values in the GGCC

Using the existing circuits we can give a circuit for the " $9x + 1$ " problem which means we have a circuit for the limit of the Quantum Generalised Collatz Conjecture Equation (equation 6.7) with $q = 9$ and $w = 1$. This would be



Flg Z30

Using the existing circuits we can give a circuit for the " $9x + 3$ " problem which means we have a circuit for the limit of the Quantum Generalised Collatz Conjecture Equation (equation 6.7) with $q = 9$ and $w = 3$. This would be representent by the circuit for one iteration



Flg Z31

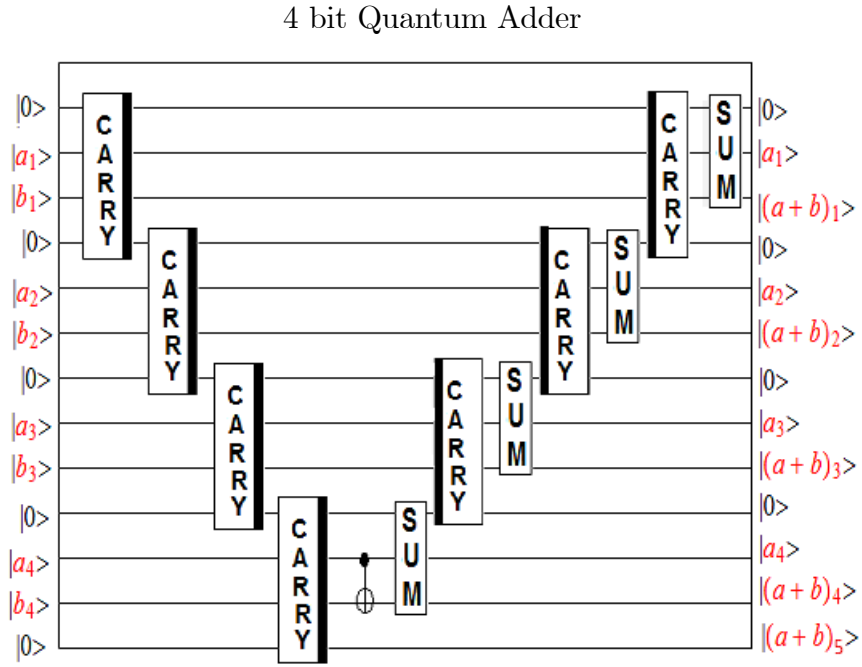
Similarly we can give a circuit for the $qx + w$ problem where q and w are powers of three using the circuit above. Actually we can give a circuit using the above ideas using the base three expansions, in other words decompose q and w into a sum of powers of three, of an arbitrary q and w (so q and w are not restricted to power of three here). But there is a more straightforward way to do it for arbitrary q and w . In the next section we will give quantum versions of our circuit given above by substituting quantum gates for classical gates. The quantum circuits are consequently logarithmic in depth regarding the value of w .

Section B of Appendix-Quantum Version of Circuits Above

We will start by giving the quantum version of the circuit for the $3x + 1$ problem (i.e. when $q=3$ and $w=1$) where x is a 4 bit number. Then we will give a quantum circuit for the $9x + 1$ problem (so we have $q = 9$ and $w = 1$) and where x is a 4 bit number. Then we will give a general circuit for the $3x + 1$ problem (i.e. x is an arbitrary number of some length) and give a general circuit for the $9x + 3$ problem (i.e. x is an arbitrary number of some length). Then we describe how to build a quantum circuit form the $qx + w$ for odd q and w .

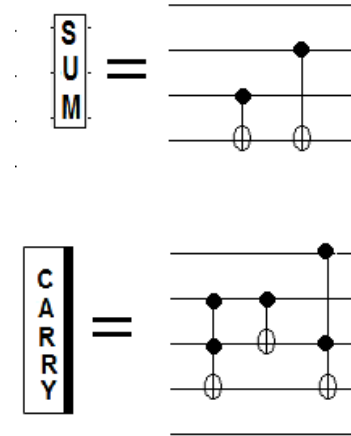
Quantum Circuit for the $3x + 1$ problem for x a 4 bit number

To describe the above we give a circuit that multiplies by 3 for when x is a three bit number because the four bit is used to accomodate for the shift by 2 described above (when the number is added to itself multiply by 3). First we give a circuit to add two 4 bit numbers -which is a extension of the circuit to add two 3 bit number given by Draper in [5]. The circuit is as follows



Flg Z32

where



Flg Z40

the bar to the left of the carry symbol means it is the reverse of the carry circuit above. We refer to the above adder which adds two numbers each of 4 bit in length with the following short hand symbol

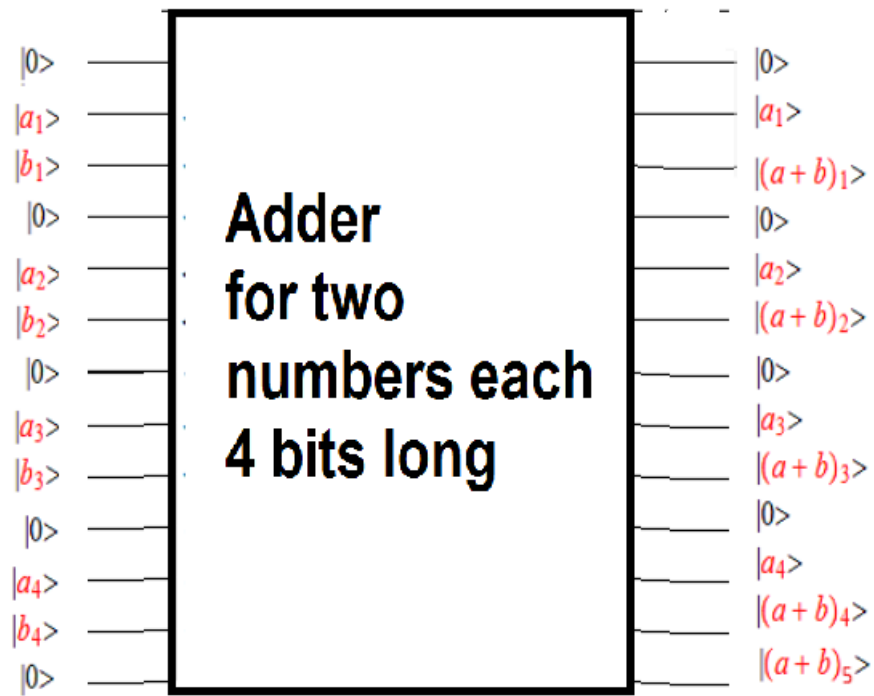
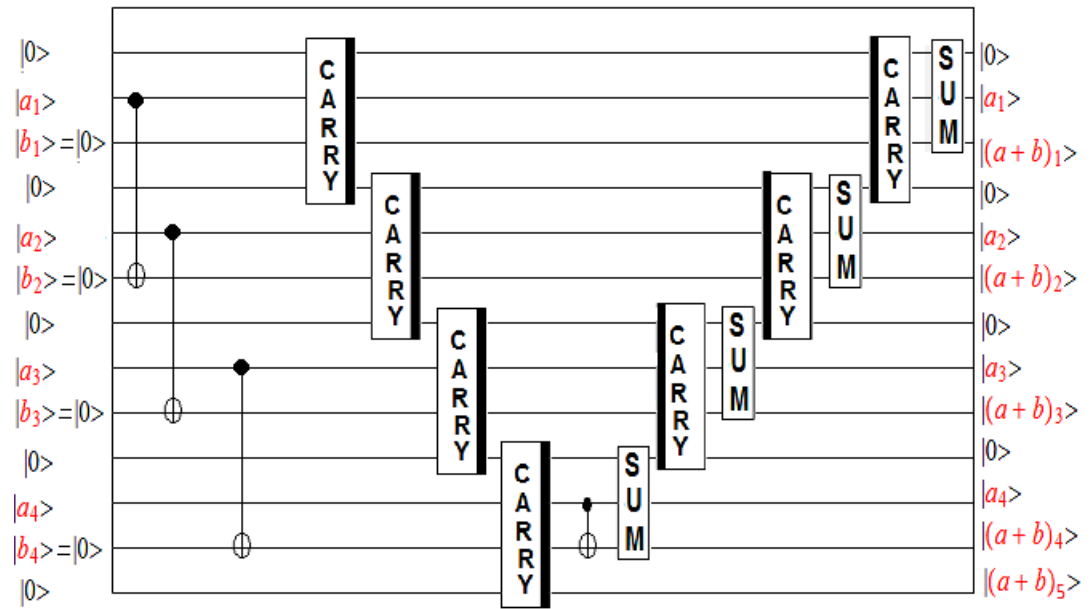


Fig Z41

Now if we added a shifted version of a 4 bit number to itself we get the multiply by three quantum circuit.



Flg Z42

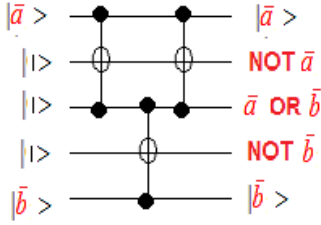


Fig Z105

The above circuit is expressed as below in shorthand

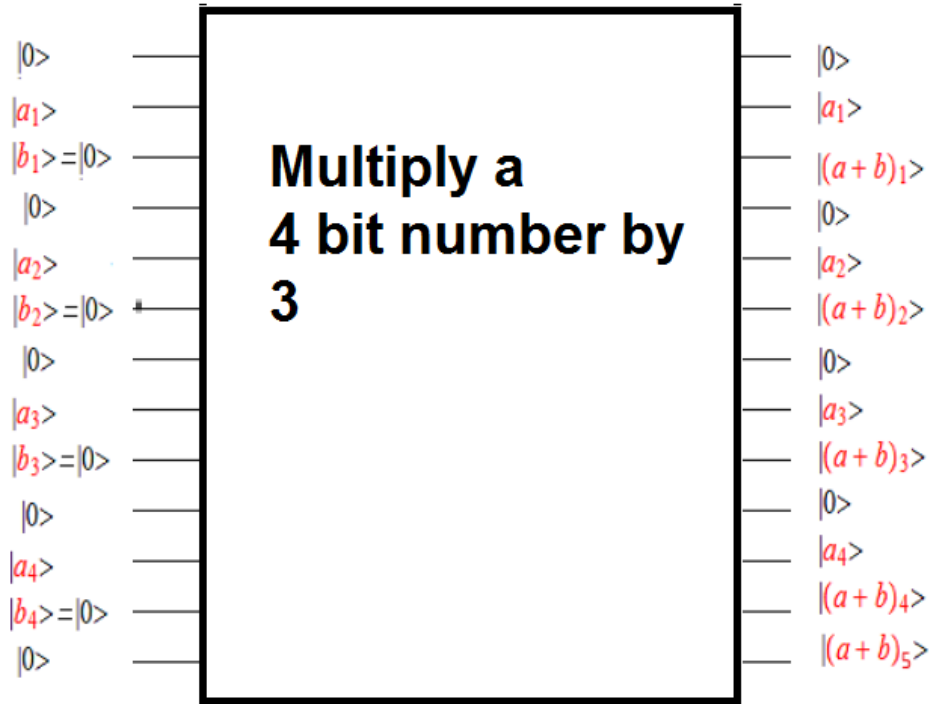


Fig Z45

Now to compute the bitwise *OR* in the equation

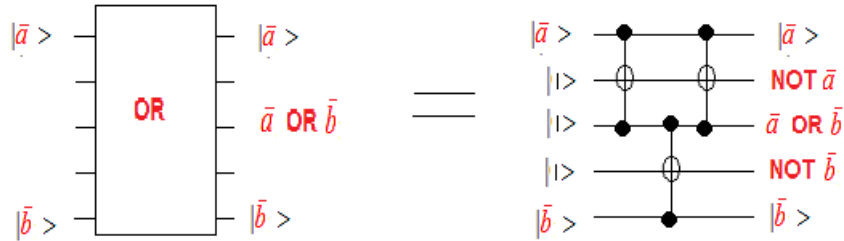
$$\Pi(\xi) = \xi \vee (2\xi) \vee (4\xi) \vee (2^3\xi) \quad (\text{A7})$$

so $\Pi(\xi)$ is the bitwise *OR* operator, $\vee$, applied p times in the above. Then

$$LSB'(\xi) = (2\Pi(\xi) \oplus (2^{2p+1} - 1)) \wedge \Pi(\xi) \quad (\text{A8})$$

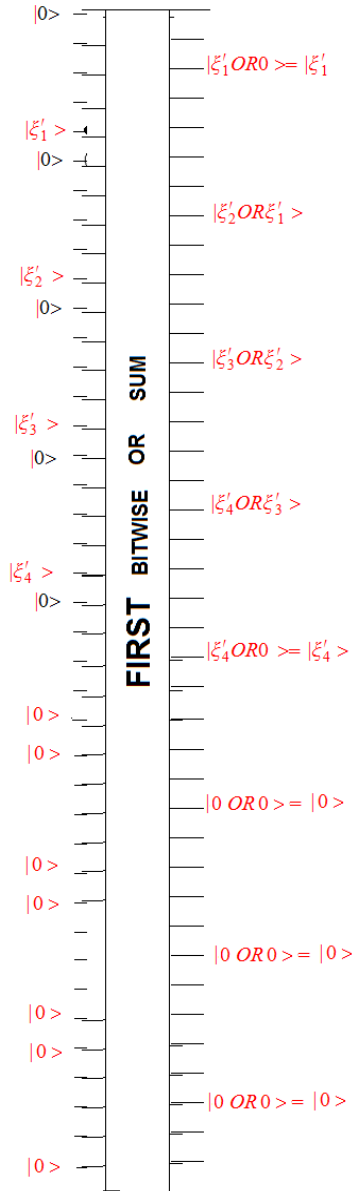
we need a bitwise quantum or gate. We use the Toffoli gate as follows the middle three inputs to the left and middle three inputs to the right can be considered as garbage here required to keep the gate reversible -for short hand we write the above gate without writing the

garbage input and output lines in other words we have



Flg Z46

also the bars on the variables above does not mean inversion-it is just simply to distinguishes the variable for clearness as the different variables with similar notation were used above. So an equation for when ξ is four bits long at most and to compute $A7$ (which results in a number that is at most 8 bits long) gives the following circuit for just computing the part $\xi' \vee (2\xi')$ (the bitwise or product of a number of itself and the shift of itself) where $\xi' = 0000\xi'_4\xi'_3\xi'_2\xi'_1$ a four bit number.



Flg Z48

similarly for just computing the part $(4\xi') \vee (8\xi')$ we use the circuit

ie. $(00\xi'_4\xi'_3\xi'_2\xi'_100) \text{ OR } (0\xi'_4\xi'_3\xi'_2\xi'_1000)$ as follows

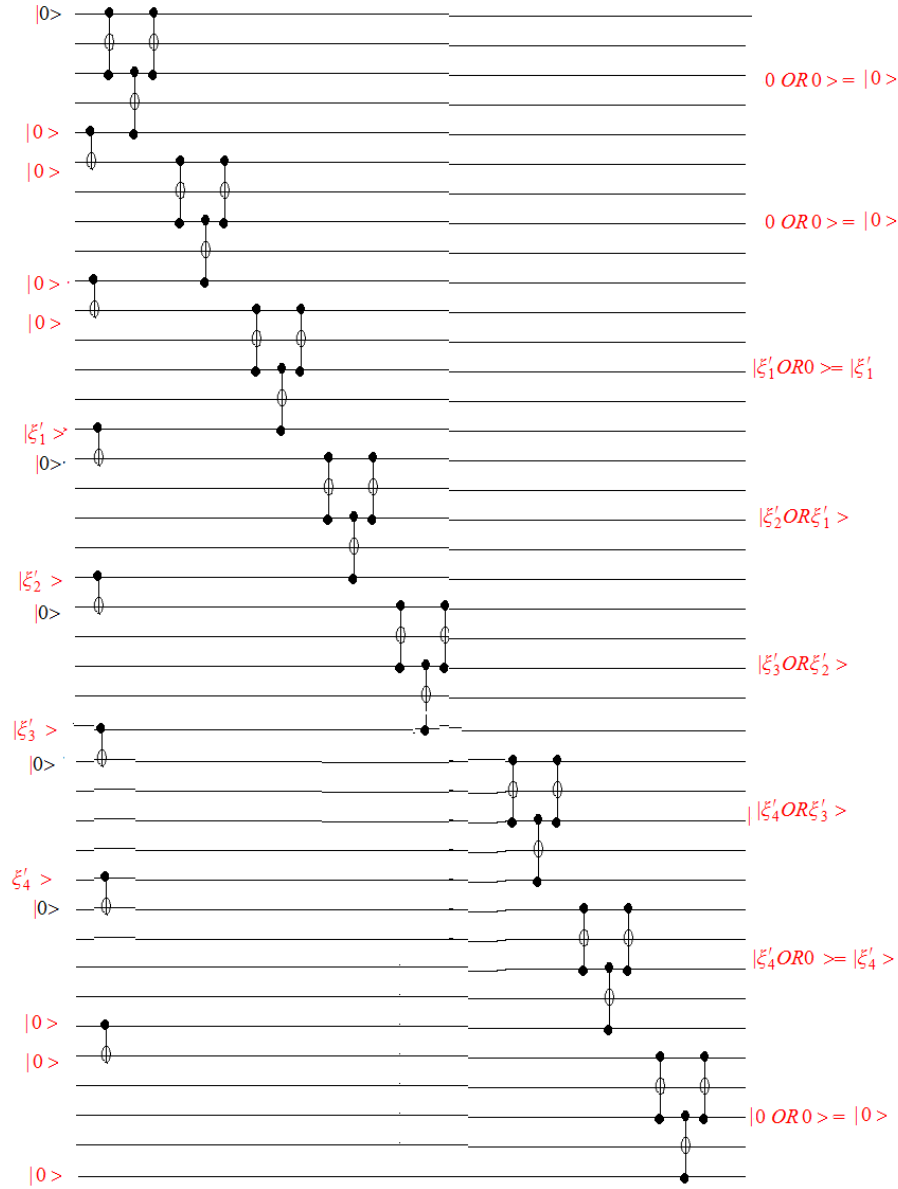
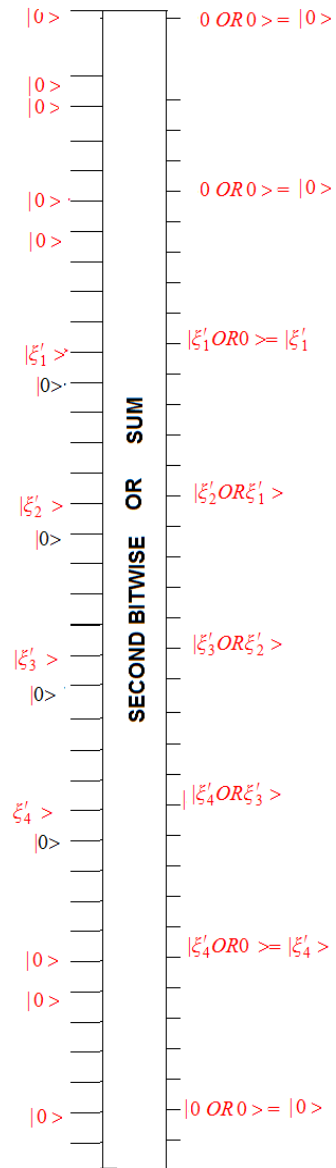


Fig Z49

we refer to the above with the short hand



Flg Z50

A circuit to compute the bitwise *OR* of two arbitrary 8bit numbers
(not a specialised version of the bitwise OR product)

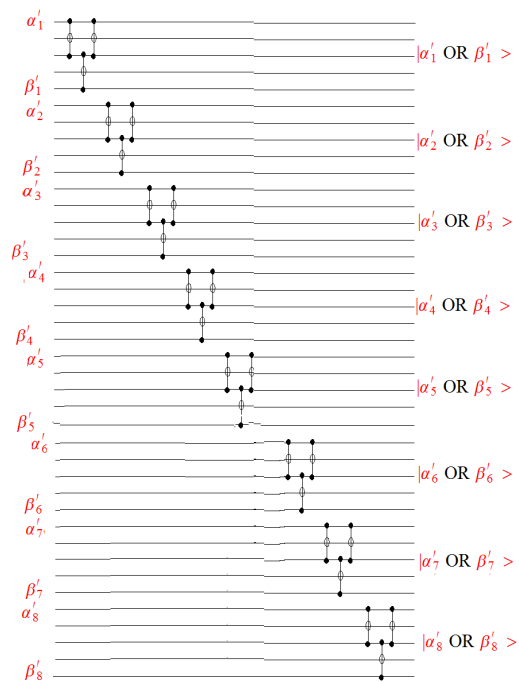


Fig Z51

We use the following shorthand symbol for above

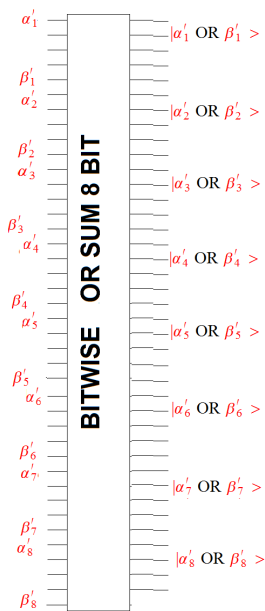


Fig Z52

Therefore a circuit that computes $\xi' \vee (2\xi') \vee (4\xi') \vee (8\xi') = \Theta$ (where Θ is an 8bit integer with the binary expansion $\Theta_8\Theta_7\Theta_6\Theta_5\Theta_4\Theta_3\Theta_2\Theta_1$ where Θ_s is some binary digit) can be constructed from the bitwise or of the circuit that computes $\xi' \vee (2\xi')$ and $(4\xi') \vee (8\xi')$ using the above circuit where $(\alpha' = \xi' \vee (2\xi'), \beta' = (4\xi') \vee (8\xi'))$ and a circuit that copies the ξ' using a series of CNOTs (which is the left most part of the circuit below) is the following circuit below

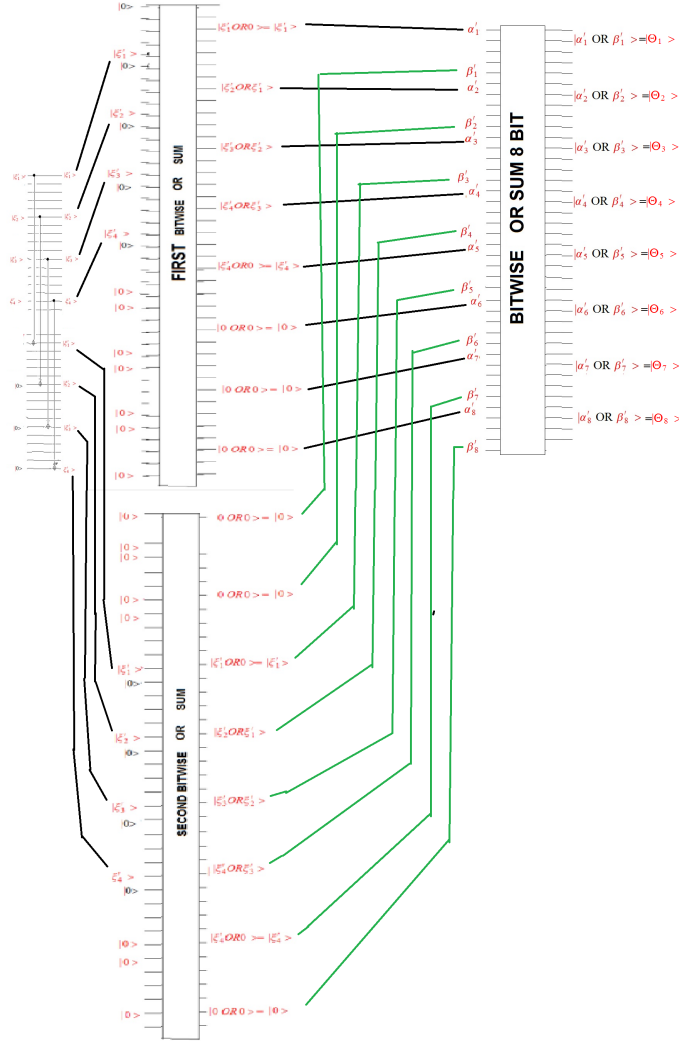


Fig Z55

the wires in the above diagram are shown in green (instead of black to avoid confusion) are wire that do not intersect the black wires where they cross but are identical in function to the black wires apart from being green. We use the following short hand for the above diagram.

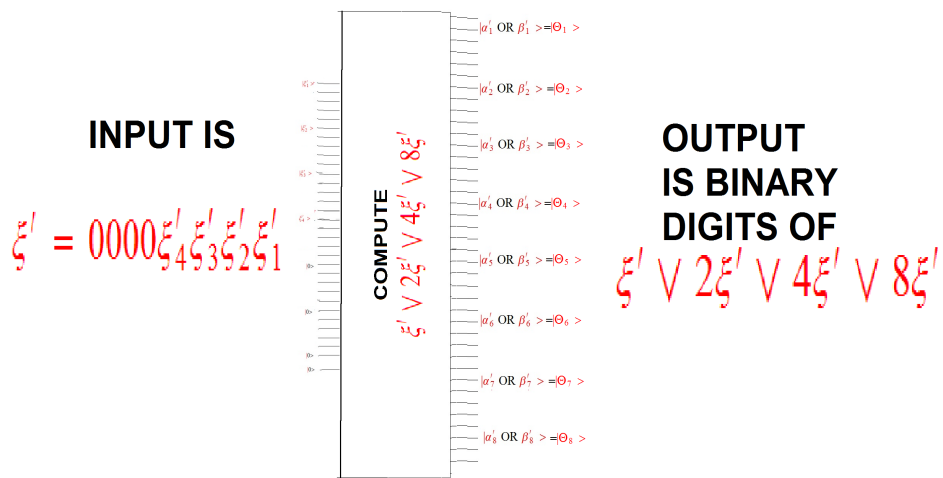


Fig Z56

the $(\oplus(2^{2p+1} - 1))$ part in equation A7 (which is $(2\Pi(\xi) \oplus (2^{2p+1} - 1)) \wedge \Pi(\xi)$) can be computed from the circuit

is equivalent to XORing with a binary number which has only $2p$ ones in its binary expansion; computing such numbers which consists of a string of ones can be done by a qubit with state $|1\rangle$ p times using the following circuit. In our example for 4 bit numbers it mean $2^9 - 1 = 111111111_2$. The number $111111111_2 = 511$ is represented in qubits with the following circuit.

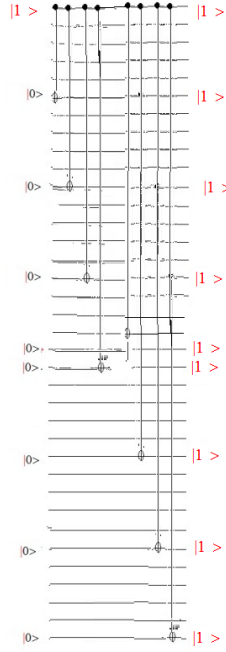


Fig Z57

we can append a circuit to xor the output of the above circuit with number J , therefore a circuit to compute $J \oplus (2^{2p+1} - 1)$ for some arbitrary 9 bit number $J = J_9 J_8 J_7 J_6 J_5 J_4 J_3 J_2 J_1$ is as follows (where the circuits

are appended is shown by green wires instead of black for clearness)

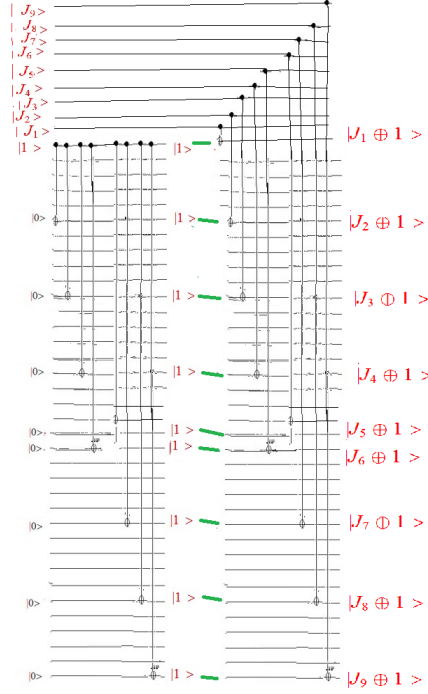


Fig Z58

we refer to the above circuit as $J \oplus (2^{2p+1} - 1)$ with $p = 4$ shown below for short hand-we do not show the ancillary input or the garbage inputs and outputs in the shorthand for the above circuit.

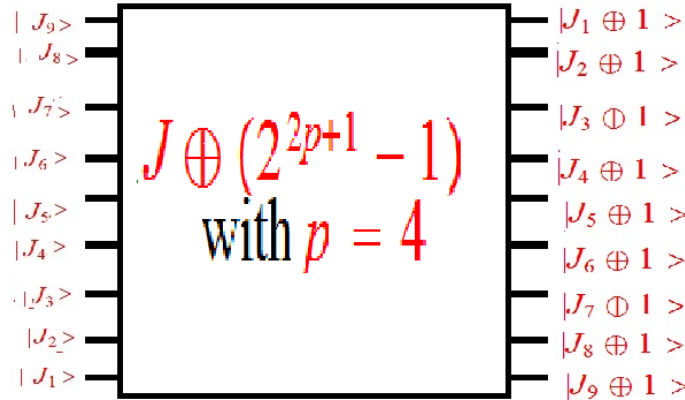
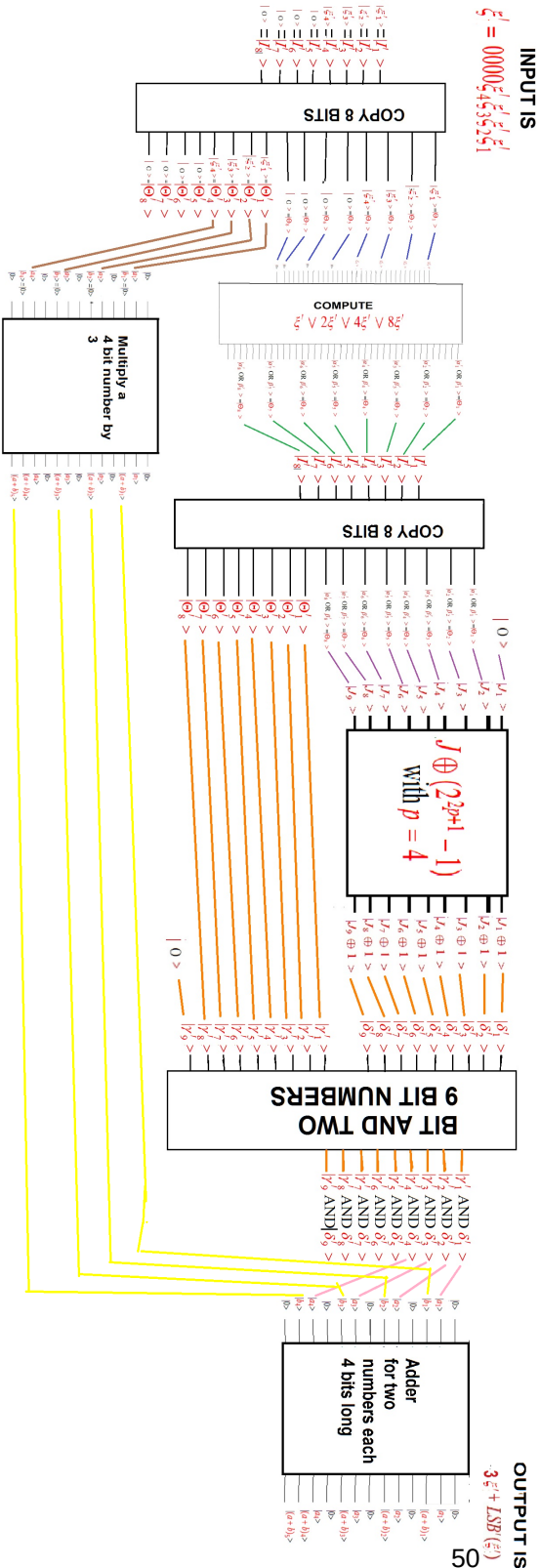


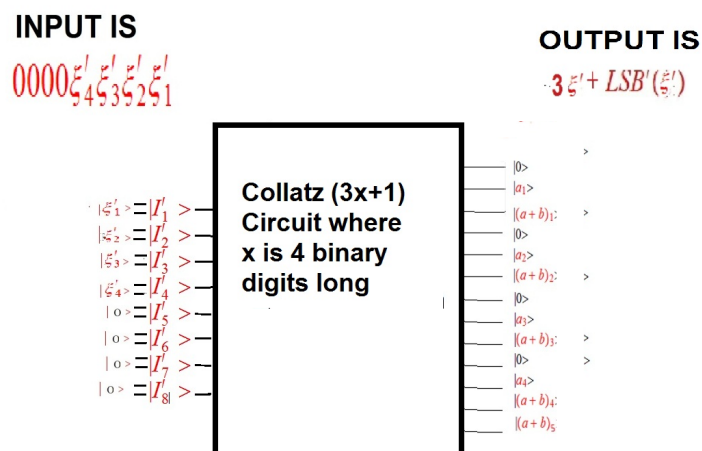
Fig Z59

Hence a quantum circuit to compute one iteration of the Collatz of $3x + 1$ where x is a four bit number (it is a quantum version of one of the classical circuits in the previous section) is as follows (Fig.1)

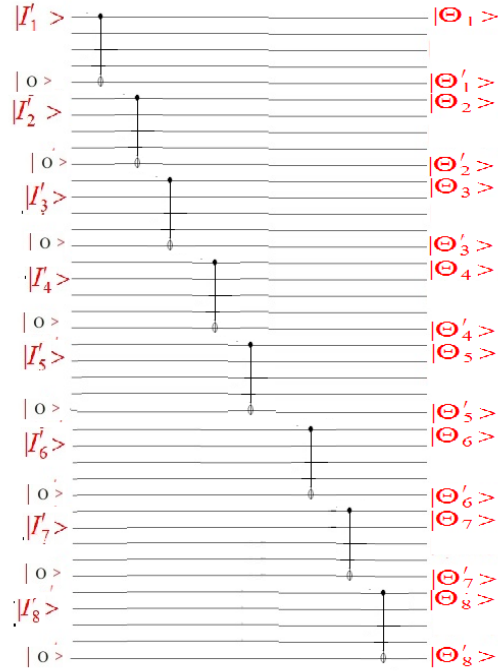
Fig.1 Quantum Circuit for one(4 bit iteration)of 3x+1 or the Collatz Iteration



we refer to the above Fig 1. circuit shorthand above as the following
fig.2

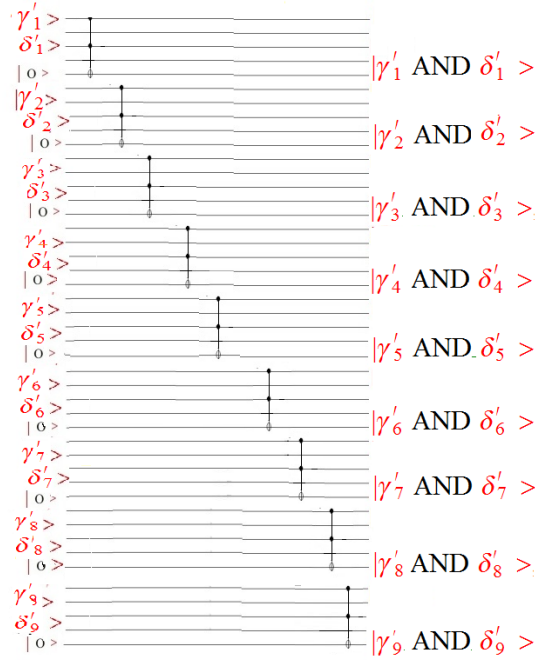


in Fig.1 the "COPY 8 BITS" subcircuit takes an 9 bit number γ' and δ' as inputs (which has the binary expansion $I_8 I_7 I_6 I_5 I_4 I_3 I_2 I_1$) as input and output two copies of it as the numbers Θ and Θ' which has the binary expansions $\Theta_8 \Theta_7 \Theta_6 \Theta_5 \Theta_4 \Theta_3 \Theta_2 \Theta_1$ and $\Theta'_8 \Theta'_7 \Theta'_6 \Theta'_5 \Theta'_4 \Theta'_3 \Theta'_2 \Theta'_1$ respectively; actually $\Theta_8 \Theta_7 \Theta_6 \Theta_5 \Theta_4 \Theta_3 \Theta_2 \Theta_1$ is just the input $I_8 I_7 I_6 I_5 I_4 I_3 I_2 I_1$ relabelled (i.e. $\Theta_1 = I_1, \Theta_2 = I_2$ etc



Flg Z60

in Fig.1 the "BITAND TWO 9 BIT NUMBERS" subcircuit takes an 8 bit number I as input (which has the binary expansion $I_8 I_7 I_6 I_5 I_4 I_3 I_2 I_1$) as input and output two copies of it as the numbers Θ and Θ' which has the binary expansions $\gamma'_9 \gamma'_8 \gamma'_7 \gamma'_6 \gamma'_5 \gamma'_4 \gamma'_3 \gamma'_2 \gamma'_1$ and $\delta'_9 \delta'_8 \delta'_7 \delta'_6 \delta'_5 \delta'_4 \delta'_3 \delta'_2 \delta'_1$ respectively;and out puts the 9 bits $|\gamma'_1 \text{ AND } \delta'_1 >$, $|\gamma'_2 \text{ AND } \delta'_2 >$, $|\gamma'_3 \text{ AND } \delta'_3 >$, $|\gamma'_4 \text{ AND } \delta'_4 >$, $|\gamma'_5 \text{ AND } \delta'_5 >$, $|\gamma'_6 \text{ AND } \delta'_6 >$, $|\gamma'_7 \text{ AND } \delta'_7 >$, $|\gamma'_8 \text{ AND } \delta'_8 >$, $|\gamma'_9 \text{ AND } \delta'_9 >$



Flg Z67

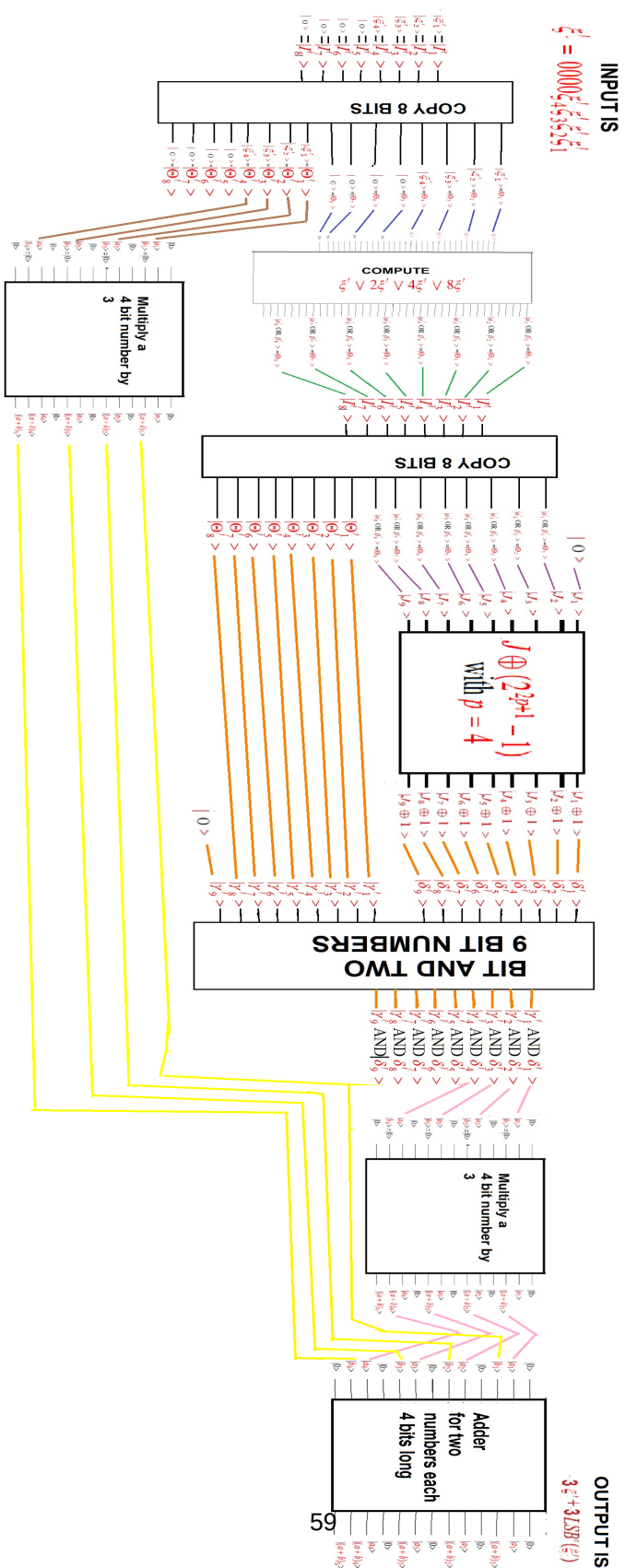
In fig 1 the wires are shown as different colour yellow,green, purple,orange,brown,blue show the wiring between the different subcircuits for clearness instead of black. Where the yellow and and purple wires cross means they are not connected. Some minor details are left our for clearness for examples the input for the $|0 >$ for the "Adder for two numbers each 4 bits long" can be taken as the $|0 >$ output from the "Multiply 4 bit number by 3" sub circuit. But the essential detail is in fig.1 describing how a quantum circuit is built for iterating the $3x + 1$ /Collatz iteration (where x is a four bit number, so it can be used to prove the for $x = 5$ is true for the $3x + 1$ /Collatz conjecture) . We can chain copies of circuit sketched out in the Fig. 1 to obtain two iteration as shown below(sketched).

So in general we have the quantum version of the classical version of the circuits (for ω iterations of the $3x + 1$ /Collatz iteration) as show in Fig. 3 below

for a four bit number-this can prove $x = 2$ fall into the known cycle for example of the $3x + 1$ problem

An x (with Bit length 4) version for the $3x + 3$ problem (where $q = 3$ and $w = 3$) in the generalised Collatz Conjecture will have the following

(sketched out) circuit Fig. 4 for one iteration



An x (with bit length 4) version for the $9x + 3$ problem (where $q = 9$ and $w = 3$) in the generalised Collatz Conjecture will have the following (sketched out) circuit Fig. 5 for one iteration

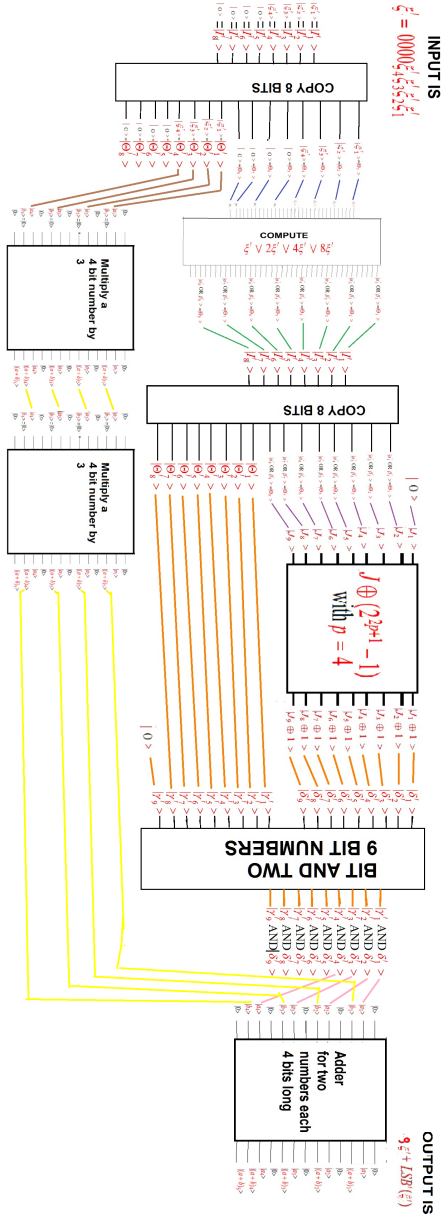


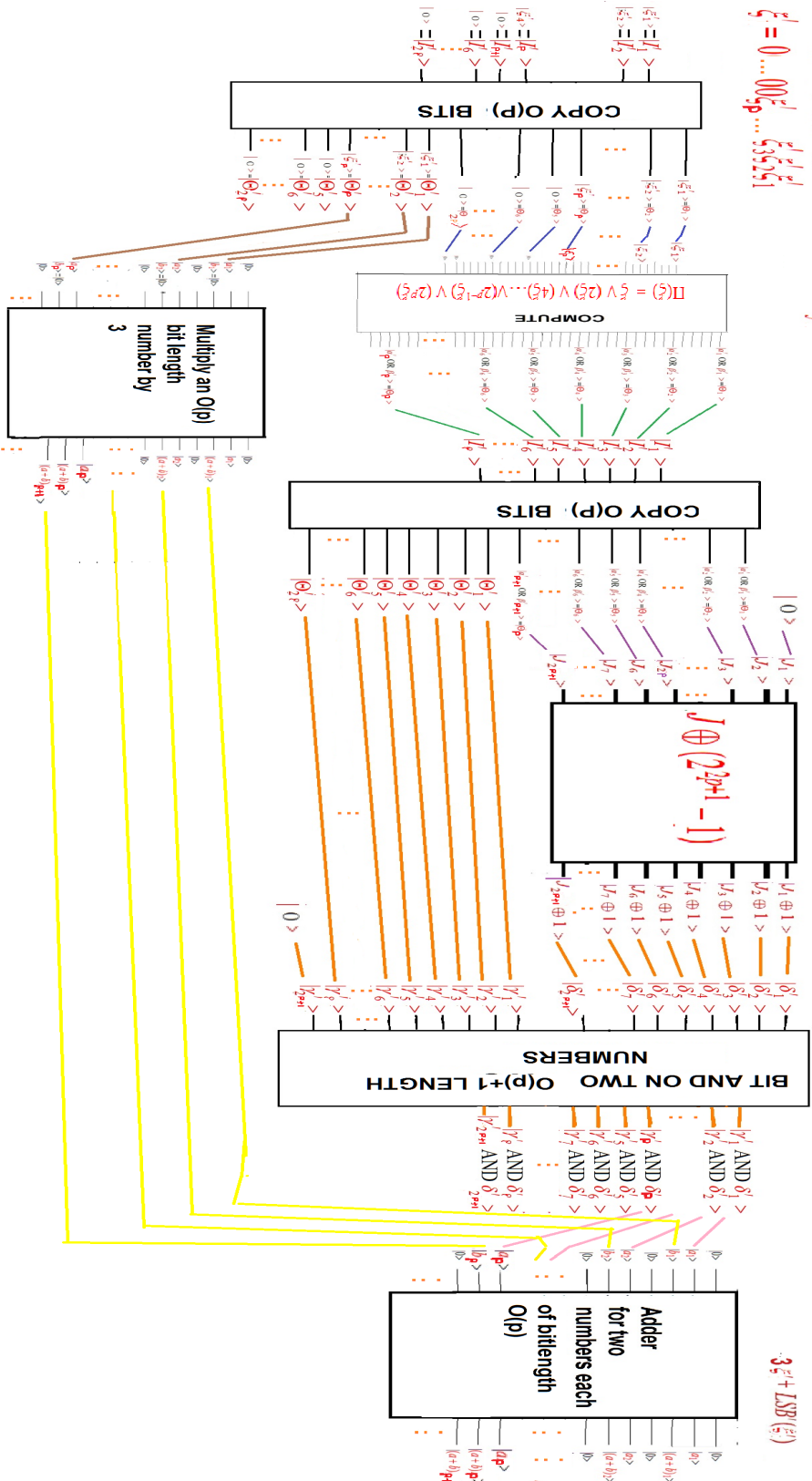
Fig 4 and fig 5 represent one iteration for a quantum circuit for the " $3x + 3$ " and the " $9x + 1$ " but these circuits can be "chained" together to create a circuit like as in given in Fig.3 to prove the assumed convergence and assumed divergence of the " $3x + 3$ " and " $9x + 1$ " problem respectively-with an adapted subcircuit for the subcircuit labelled q - for

some four bit integer x -a similar remark holds for any values q and w in the generalised Collatz conjecture in the " $qx + w$ " problem.

"By generalised the circuit in Fig. 1 so that each subcircuit deals with a p bit arbitrary number, this can be done by generalising the above subcircuits to deal with p bits we get a quantum circuit that can verify the Collatz conjecture for any arbitrary p and this generalised quantum circuit has expected polynomial time and polynomial space complexity. It is sketched out in Fig 7 below.-it is for an input; a similar comment about "chaining" applies here to Fig.7 below, to give a quantum circuit that can compute the $3x+1$ /Collatz iteration for any arbitrary bit length number x .The final circuit output of below would pass through an p bit version of the circuit q (not shown in circuit diagram below); the "chaining" is discussed below and in Fig. 8

INPUT IS $2p$ bit length number $\xi^{(1)}$

$$\xi^{(1)} = 0 \dots 00 \xi_p^{(1)} \dots \xi_1^{(1)} \xi_0^{(1)}$$



If we refer to the above circuit with the short hand

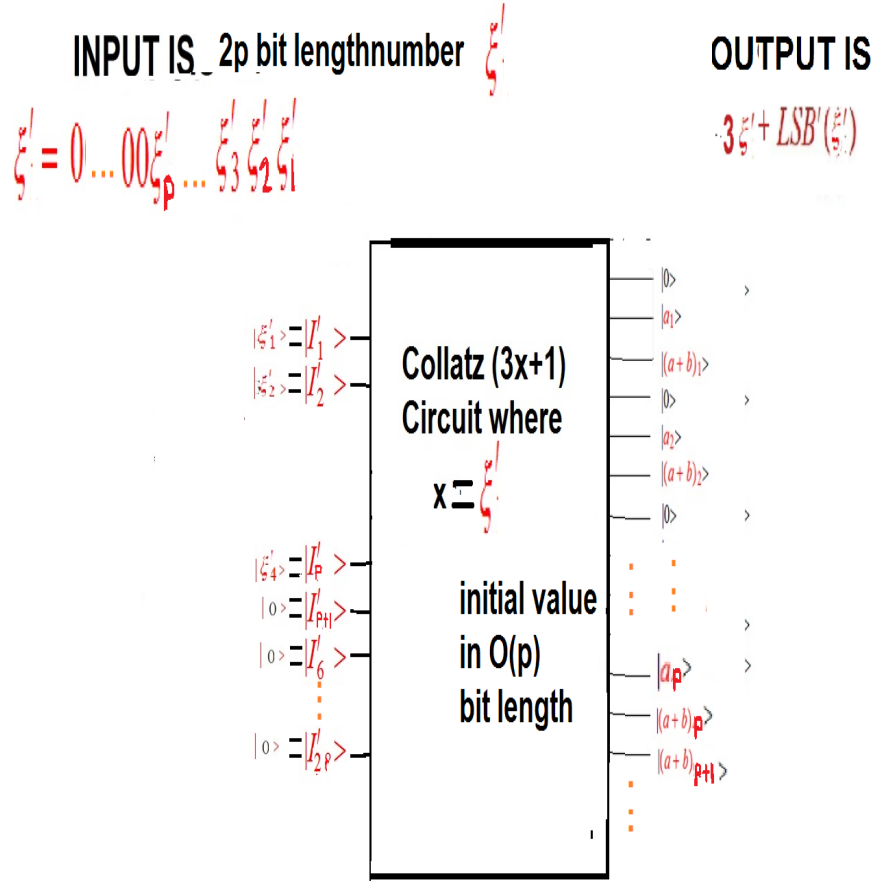


Fig Z95

So similar to the above fig 3 (for when x is four bits) we get the following circuit Fig. 10 below when we "chain ω " copies together. The green lines show how the wires are connected between each copy of the above -the green line represents $O(p)$ lines to be connected or the three dots indicate the representation of $O(p)$ objects (a group of input or output wires or a group interconnecting green wires). The final output of the circuit is some number $O'(p)$ for some (Big-Oh) linear function of p ; $O(p)$ is some linear function of p in the circuit. Fig. 10 is a description/sketched circuit for any initial value x in the $3x + 1$ problem for ω iteration (where ω is expected to be polynomial in the initial value $x = \xi = 0 \dots 00\xi_p \xi_{p-1} \dots \xi_3 \xi_2 \xi_1$, where x is considered to have p leading zeroes in its representation (which of course does not change the value of x e.g. $x = 1101_2$ is considered to be same as $x = 00001101_2$ here).

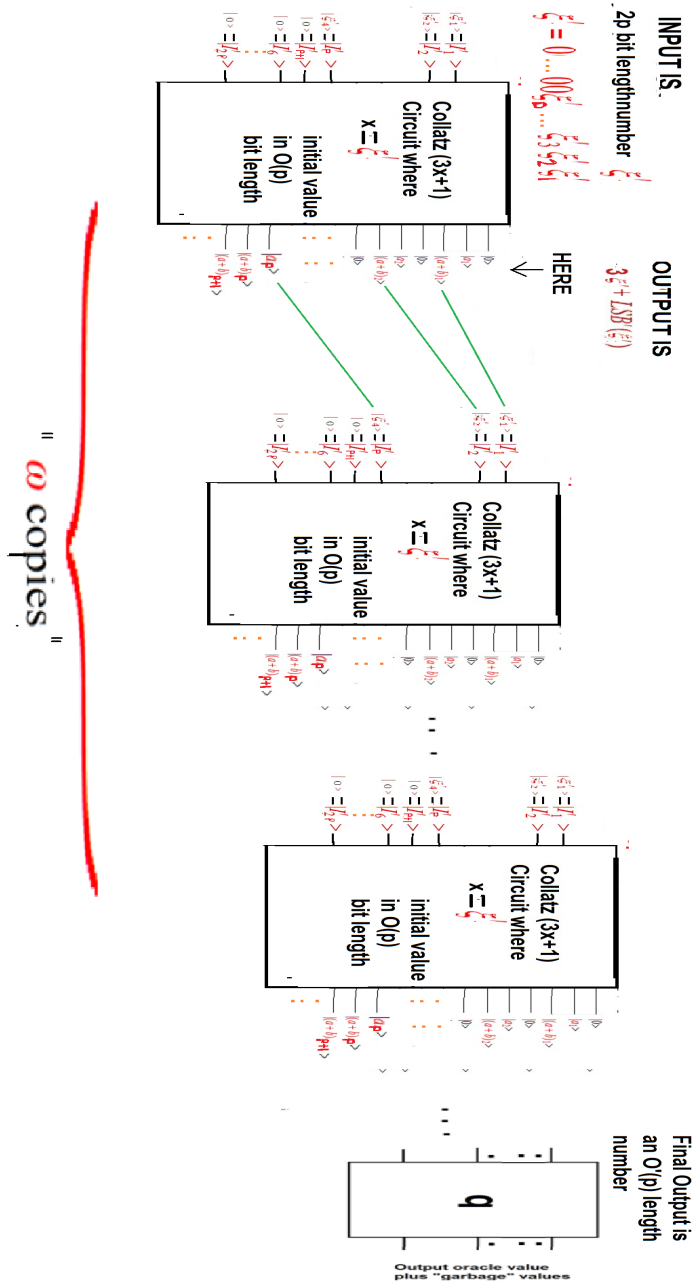
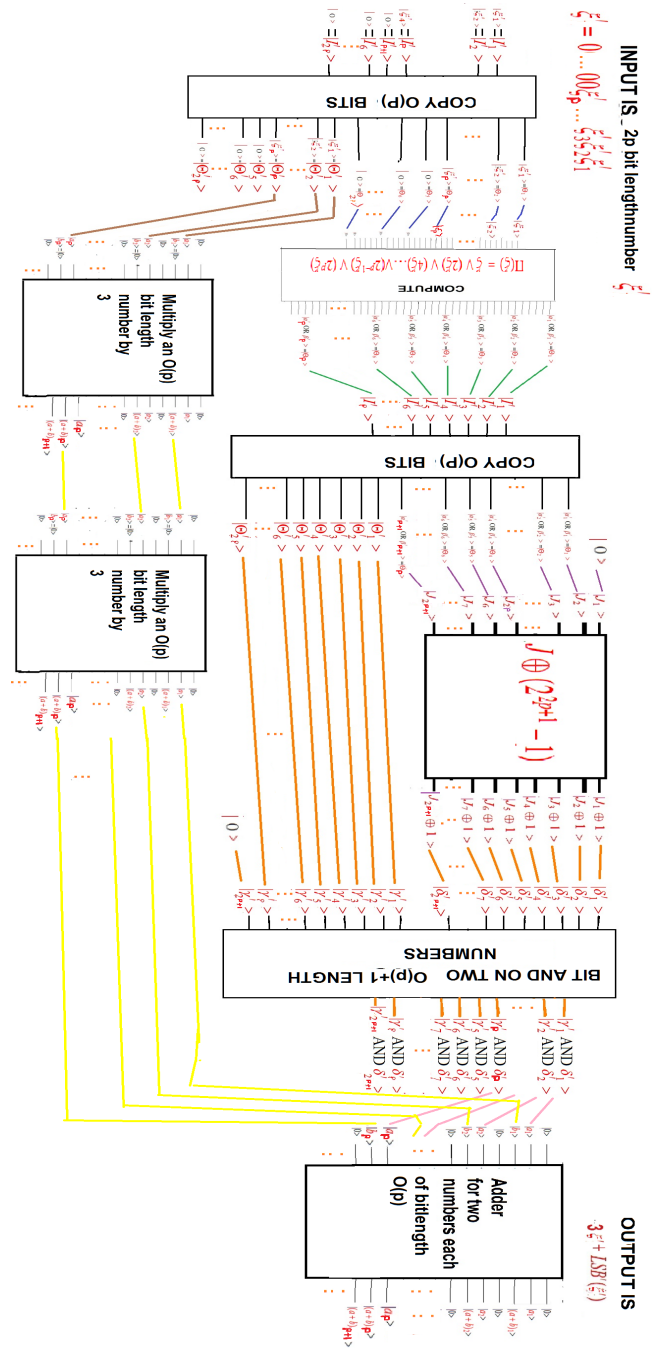


Fig. 10 is below

We can give versions of the above circuit (for one iteration of it, then for ω iterations) for the $qx + w$ problem, in Fig. 11 and Fig. 12 below where $q = 9$ and $w = 1$ -we can use an existing multiply circuit such as the given in [6] adapted to use here to compute the " qw part in the $qx + w$ problem.". But we use our multiply by 3 circuit already described. Actually, instead of using the quantum multiplication described in [6], we can build a quantum circuit to compute qx using our existing multiply by 3 circuit above, an addition and the base three expansion of the number q (it is like computing a weighted sum here in powers of 3). Note that the final sub circuit q' is different for this problem compared to the above analogous sub circuit q which is described in Fig. 10 because the $9x + 1$ problem is expected to diverge.

Fig. 11



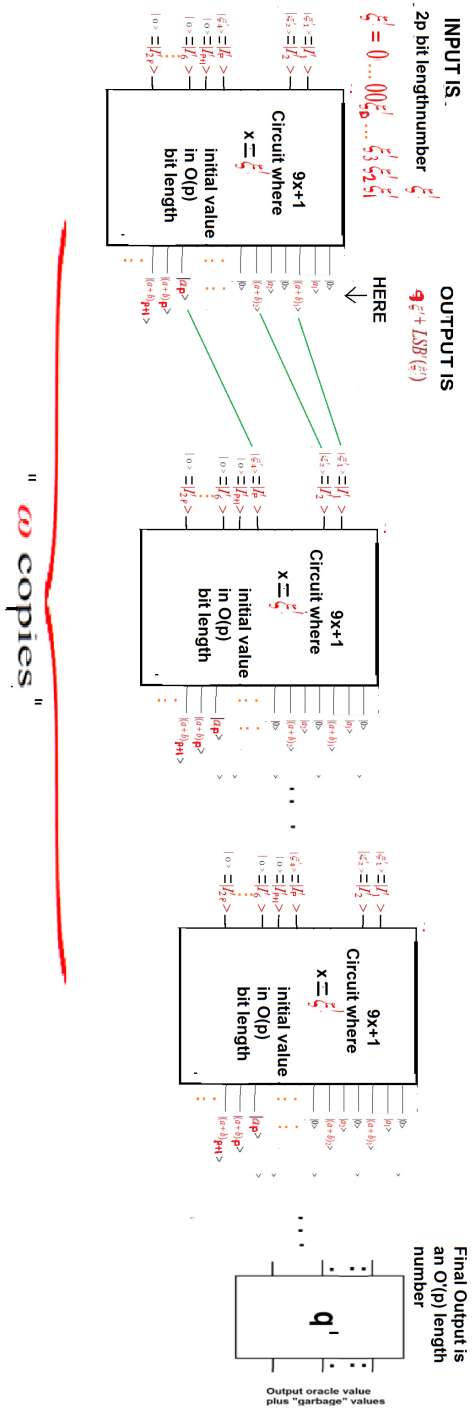
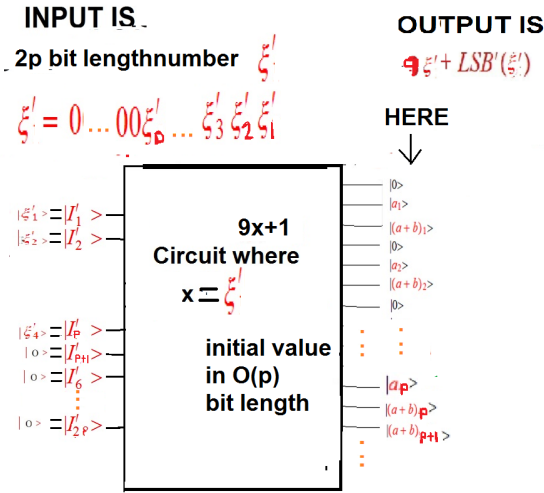


Fig. 12

Where we use the short hand (Fig. 13 below) for Fig. 10 which is used in the above fig for the $9x + 1$ problem



So this paper describes how to construct a quantum circuit for any values $qx + w$, in the generalised Collatz Conjecture, for an arbitrary starting value, and some examples are given above.

Appendix B: A Symmetric $(q, w) = (7, 7)$ Case Study: Conjectural Divergence at $x_0 = 29$

This appendix records an investigation of the symmetric generalised system $f_{7,7}(x) = 7x + 7\text{LSB}(x)$, examining a natural proof strategy for divergence at $x_0 = 29$ built on a residue-class invariance argument. We verified the derivation computationally before including it, found that the central step does not hold, and report both what is established and what remains open, rather than present it as a theorem.

What is established

Write $x = \text{LSB}(x) \cdot y$ with y odd. Since $\text{LSB}(7m) = \text{LSB}(m)$ for any integer m (Proposition 3, 7 being odd), the odd core of $f_{7,7}$ decouples exactly as

$$y_{k+1} = 7 \cdot \frac{y_k + 1}{2^{v_k}}, \quad v_k = \nu_2(y_k + 1), \quad (\text{B200})$$

matching the derivation in Section 3.4. Two further facts are and correct: (i) $7 \mid x_k$ for every $k \geq 1$ (immediate by induction from $x_{k+1} = 7(x_k + \text{LSB}(x_k))$), so $f_{7,7}$'s trajectories can never reach an exact power of two the way the classical map's do – “convergence” in the sense of Proposition 1 is algebraically impossible here; (ii) $y^* = 7$ is an exact fixed point of the odd-core recurrence ($7 + 1 = 8 = 2^3$, so $y^* \rightarrow 7 \cdot 1 = 7$), consistent with Proposition 1 since $q + w = 14$ is not a power of 2 but $u^* = 7$ satisfies $qu^* + w = 56 = 2^3 \times 7$.

For $x_0 = 29$, direct computation gives odd-core values

$$29 \xrightarrow{v=1} 105 \xrightarrow{v=1} 371 \xrightarrow{v=2} 651 \xrightarrow{v=2} 1141 \xrightarrow{v=1} 3997, \quad (\text{B201})$$

which we confirmed matches $v_k = \nu_2(y_k + 1)$ exactly.

Where the argument breaks

The natural argument one might build from this concludes, from this one 5-step block, that $y_k \equiv 29 \pmod{128}$ recurs with period 5 *for all* k , and that $v_k \leq 2$ holds forever, giving geometric divergence. We tested this directly: of 2000 other integers in the same residue class $y \equiv 29 \pmod{128}$, 1984 do *not* reproduce the valuation pattern $(1, 1, 2, 2, 1)$ or return to 29 $\pmod{128}$ after five steps (e.g. $y = 157 \equiv 29 \pmod{128}$ gives valuations $(1, 1, 2, 2, 3)$ and lands on 81 $\pmod{128}$). So the residue class mod 128 does not determine the future trajectory, and the claimed period-5 invariant is false as a general statement – it only re-verifies the one instance already used to construct it. Directly simulating the actual $x_0 = 29$ trajectory confirms this concretely: a valuation of $v_k = 3$ occurs at $k = 20$, and larger values (up to 5, in the range checked) occur later, so $v_k \leq 2$ does *not* hold for all k , contrary to what a geometric divergence bound of this kind requires (a uniform cap $v_k \leq V < \log_2 q$ for all k).

What remains open, and what the numerical evidence shows

We simulated the $x_0 = 29$ trajectory for 2000 steps: no cycle was found (no repeated odd-core value), and the growth rate settled near 0.82 bits/step, close to the heuristic prediction $\log_2 7 - \mathbb{E}[\nu_2] \approx 2.807 - 2 = 0.807$ bits/step from the average-drift heuristic above. This is consistent with divergence and is the kind of evidence that makes the conjecture plausible, in the same way extensive verification supports CC itself ; but it is heuristic and finite, not a proof, for exactly the

reason bulk verification of CC is not a proof of CC. We record the following as a conjecture rather than a theorem:

- **Conjecture (open).** The odd-core trajectory of $f_{7,7}$ started at $y_0 = 29$ has no periodic cycle and $y_k \rightarrow \infty$.

A proof would need either an argument that holds for arbitrarily large moduli (not just one 5-step block checked against itself), a density/ergodic argument over the full residue tree in the style of existing heuristic treatments of generalised Collatz maps, or a computer-assisted search ruling out cycles up to a very large bound combined with a rigorous tail bound on valuation excursions – none of which is attempted here.

Why no worst-case argument can resolve this: the $q = 1$ boundary

There is one genuinely rigorous result adjacent to this problem, worth recording precisely because it shows exactly where rigor stops rather than leaving that boundary implicit.

Proposition 1. *For $q = 1, w = 1$, every trajectory of $g_{1,1}(x) = x + \text{LSB}(x)$ eventually reaches, and permanently remains at, an exact power of two.*

Proof. Write $x = 2^k m$ with m odd. Then $x + \text{LSB}(x) = 2^k(m + 1)$, and since m is odd, $m + 1$ is even, so the new odd part is $(m + 1)/2^{\nu(m+1)} \leq (m + 1)/2$. For $m > 1$ this is strictly less than m , so the odd-part sequence is strictly decreasing while it exceeds 1. A strictly decreasing sequence of positive odd integers reaches 1 in finitely many steps, and $m = 1$ is then a fixed point ($1 + 1 = 2 = 2^1$, odd part returns to 1). Once the odd part is permanently 1, x is permanently a power of two. $\square$

We verified this against 20,000 random starting values, confirming convergence in every case and confirming the odd part is non-increasing at every single step throughout, the case $g_{1,1}(x)$ is known to converge (since it is equivalent to the "x+1" problem).

Proposition 2. *For every odd $q \geq 3$, the worst-case single step of the odd-core map (minimum possible valuation, $v = 1$) increases the odd part already at $m = 1$: $q(1 + 1)/2 = q > 1$.*

Proof. Immediate substitution. $\square$

So Proposition 1's technique is unknown to extend to $q = 3$ (the classical Collatz case) or $q = 7$ (this appendix's case), it is structurally unavailable for either, and for the same reason in both cases: the one inequality the whole proof rests on reverses direction the moment q leaves 1. This is a precise, provable account of why worst-case monotonicity cannot decide either the classical convergence conjecture or this divergence conjecture, rather than an admission that we simply did not find the right argument.

The obstruction is not a vacuous worst case, either. For $(q, w) = (7, 7)$ we searched starting values for how many *consecutive* $v_k = 1$ steps (the worst case for divergence – weakest possible shrinkage) actually occur, and found $y_0 = 1637$ sustains $v_k = 1$ for 12 consecutive steps, with growth over that run matching $(7/2)^{12}$ almost exactly (3,382,110 actual against 3,379,221 predicted, the small gap being the lower-order "+1" correction the bound $q(m + 1)/2$ carries). This is consistent with, and explained by, the unnormalised recursion $z \mapsto 7(z + 1)/2$ (dropping the valuation-stripping) having a fixed point at $z^* = -7/5$ in the 2-adic integers $\mathbb{Z}_2$ (well defined since 5 is a unit there); $z^* \equiv 5 \pmod{16}$, matching $1637 \equiv 5 \pmod{16}$ exactly, and standard 2-adic shadowing of a fixed point of this kind is the reason to expect such runs at every length, not only

the one found by search ; though we have verified this only computationally up to length 12 and are not asserting a proof that it extends to all lengths.

None of this proves the Conjecture above; it proves something narrower : that the probable natural first line of attack (bound the worst case, conclude by monotonicity) is not just insufficiently here but provably inapplicable, for exactly the reason it is also provably inapplicable to the 80-year-open classical case. Whatever eventually resolves either conjecture will have to be a different kind of argument.

Appendix C: A Sliding-Window Variant, and What a Captured Cycle Would Imply for Query Complexity

Section 5.1’s algorithm fixes the search domain at $D = \{1, 3, 5, \dots, 2^{\alpha+1} + 1\}$, starting at 1. This appendix records two things left out of the main construction: a natural generalisation that starts the window anywhere rather than always at 1, and a consequence, worth stating precisely, of what happens to the algorithm’s query complexity if a nontrivial cycle actually falls inside whichever window is chosen. Neither changes anything in the body of the paper; both are additions to it that give a second more general quantum algorithm.

C.1 The sliding window

Fix an arbitrary odd integer $s \geq 1$ and define

$$D_s = \{s, s + 2, s + 4, \dots, s + 2^{\alpha+1}\}, \quad (\text{C200})$$

a window of the same size $N = 2^\alpha$ as before, now starting at s instead of 1. Every part of Section 5’s construction, the oracle $O(t, x)$, the Grover operator, the quantum-counting procedure, the four-output classification, carries over to D_s with no change whatsoever beyond replacing D by D_s throughout: nothing about the oracle or the amplitude-amplification machinery depends on where the window starts. This is a deliberately lightweight generalisation, not a new algorithm, and it was already gestured at, informally, in the source main paper builds on (“sliding a window of width 2^Δ over some interval”) without being formalised above. The reason to formalise it at all: since classical verification already covers $[1, 2^{68}]$ from Barina’s paper, running Section 5’s algorithm at $s = 1$ is redundant, any window actually worth searching quantum-mechanically should have s at or beyond the classically verified bound.

C.2 How large k would be if a cycle is captured

Section 5.4’s outcome 3 implicitly treats a discovered counterexample as a single flagged point, but this is not the generic case. If x lies on a nontrivial cycle of period p , its trajectory never converges, so by Proposition 1’s contrapositive $O(t, x) = 1$ for every choice of t , and this holds for *every* element of the cycle, not just one. If D_s happens to contain c of the cycle’s p elements (plus, once t is large enough to reach the cycle from a transient start, any elements of D_s whose trajectories feed into it), the true count satisfies $k \geq c$, not $k = 1$. Since the Hercher bound already cited in Section 5.4 gives $p \geq 1.375 \times 10^{11}$ for any nontrivial cycle, c could in principle be very large if a window is chosen (or happens) to overlap a substantial fraction of such a cycle, and Eliahou’s period constraint (also in Section 5.4) further restricts which periods p are even structurally possible, which could in principle help in guessing where to look, though we do not develop that further here.

C.3 Consequence for the number of Grover iterations

The iteration count from Section 5.1, $H = \lfloor \frac{\pi}{4} \sqrt{N/\hat{k} - \frac{1}{2}} \rfloor$, decreases as $\hat{k}$ increases, so capturing a large chunk of a cycle makes the search *easier*, not harder, in query count. Concretely, for a window of size $N = 2^\alpha$ that happened to fully capture a minimal-length cycle ($k = p = 1.375 \times 10^{11}$): **Table C1**

α	H at $k = 1$ (needle in haystack)	H at $k = p$ (full capture)
37	2.9×10^5	0.79
40	8.2×10^5	2.2
50	2.6×10^7	71
68	1.3×10^{10}	3.6×10^4
100	8.8×10^{14}	2.4×10^9

Two things are worth being precise about here, since the table may mislead in either direction if read too quickly. First, the dramatic drops at $\alpha = 37, 40$ are inside the *already classically verified* range ($< 2^{68}$) , no counterexample can exist there, so those columns illustrate the scaling, not a usable regime. Second, at the α values that are actually useful (≥ 68), full capture of a minimal-length cycle still leaves H large (3.6×10^4 at $\alpha = 68$, growing fast beyond that) , reduced by many orders of magnitude from the $k = 1$ case, but not driven to $O(1)$ or to anything that could fairly be called polynomial in α on its own once α is in the regime that matters. Whether it is polynomial or exponential in α depends, exactly as anticipated, on how much of the window a real cycle would occupy at that scale, which is not known.

C.4 Two limits on this idea

- **No localisation heuristic exists.** Hercher’s and Eliahou’s bounds constrain the *length* and *period structure* of a hypothetical cycle, not its *location*: nothing in either result, or anywhere else in this paper, gives a principled way to choose s so that D_s is more likely than any other window of the same size to contain a cycle. Absent such a heuristic, D.3’s favourable scaling is a best-case statement about what would happen *if* a window were chosen well, not a claim that any particular choice of s achieves it. The worst-case query complexity of the algorithm is unchanged from above.
- **This is a query-count effect, not a feasibility fix.** A larger $\hat{k}$ shrinks the *number* of oracle calls, but each call still costs $O(\alpha)$ gate depth and $O(\alpha)$ qubits for the arithmetic circuits of Section 3.6, regardless of k . The hardware bottleneck identified in Section above is driven by α (register size), not by the iteration count H , so nothing in this appendix changes that picture , it sharpens what a fortunate "window" discovery would look like in query terms, without making one more feasible to obtain.

Appendix D: The General Algorithm Beyond Collatz

The general algorithm is sketched in this section. It uses the two properties in Sections 3–5 used for the $3x + 1$ map, and generalises them for other sequences; the two properties that made the construction work. This appendix states those two properties explicitly, shows that the whole algorithm reduces, in general, to a single eigenvector-finding primitive applied to the Grover operator, and gives one example candidate (the Juggler sequence) where the recipe plausibly applies and flagging the piece that would still need to be built.

D.1 What the construction actually needs

Let $x_{n+1} = \varphi(x_n)$ be any integer recurrence given by an explicit (piecewise) formula, together with a target predicate T (“has this value shown the expected behaviour”). The construction of this paper transfers verbatim provided:

- (A)-**An efficient reversible step.** φ (or, more usefully, the t -fold iterate $\varphi^{(t)}$ directly used in the oracle) admits a reversible quantum circuit computing it from an m -bit input using $C(m)$ gates, for some $C(m)$ polynomial in m . There is no requirement that $C(m)$ be linear, or even sub-quadratic: since Grover’s own speed-up is already only quadratic in N , any fixed polynomial degree for $C(m)$ preserves the same qualitative speed-up over classical exhaustive search with the same per-step cost; it only changes the polynomial overhead multiplying it. This in the sense is something below a quadratic-time classical formula for the t -th term, not necessarily a faster one, already suffices to be faster than classical for certain iterations.
- (B)-**A checkable target.** There is an efficiently testable predicate $T(y)$ (for Collatz, “ y is an exact power of two”). used in the oracle, such that $T(\varphi^{(t)}(x))$ holding for some finite t is equivalent to x exhibiting the convergence behaviour the associated conjecture asserts; exactly the role Proposition ?? plays for Collatz.

Given (A) and (B), define $O(t, x) = 1$ if $T(\varphi^{(t)}(x))$ is false, 0 otherwise, and run Sections 5.1–5.4 unchanged: uniform superposition over a chosen domain, quantum counting to estimate the number of not-yet-converged candidates, Grover’s algorithm to exhibit one if $k_{\min} > 0$, and the same four-way outcome classification, with query complexity $O(\sqrt{N/\hat{k}})$ and per-query depth $O(t \cdot C(m))$ exactly as in Sections 5 and 7.

D.2 The general reduction to eigenvector finding

Simultaneously testing an exponential number of candidate trajectories reduces to exactly one computational primitive, independent of φ and T : build the reflection operator $G = (2|\psi_0\rangle\langle\psi_0| - I)O_\chi$ from whichever oracle (A)–(B) produce. On the two-dimensional subspace spanned by the marked and unmarked uniform superpositions, G *always* has eigenvalues $e^{\pm 2i\theta}$ with $\sin \theta = \sqrt{k/N}$; this is a fact about reflection operators built this way, used in Collatz specifically (Section 5.1, Step 2). Quantum phase estimation applied to G (Section 5.3’s Steps A–E: controlled powers of G , inverse QFT, measure) is exactly quantum counting, recovering $\hat{k}$ from G ’s eigenvalue without first needing to know it; Grover’s algorithm itself (Section 5.1’s Step 4) is the coarser special case of the same idea, amplifying overlap with an eigenvector rather than reading its eigenvalue off directly. So “in both halves” of this paper’s algorithm, counting and searching „, are, in full generality, the same eigenvector-finding primitive applied to whichever reflection operator a given (φ, T) pair produces; the Collatz-specific content of Sections 3–5 is entirely contained in constructing that one pair, not in anything downstream of it.

D.3 A candidate: the Juggler sequence

The Juggler sequence, $a_{n+1} = \lfloor \sqrt{a_n} \rfloor$ for even a_n and $a_{n+1} = \lfloor a_n^{3/2} \rfloor$ for odd a_n , is conjectured (like Collatz) to reach 1 from every starting value. Condition (B) is immediate: $T(y) = (y = 1)$ is trivial to test. Condition (A) is genuinely open here, and we are explicit that this appendix does not resolve it: unlike our Collatz’s bitwise LSB reformulation with exponential sums, which sidesteps division entirely, computing $\lfloor \sqrt{a_n} \rfloor$ or $\lfloor a_n^{3/2} \rfloor$ reversibly requires a integer square-root circuit (constructible in polynomial gate count by standard digit-by-digit or Newton-iteration methods, but not via the kind of single closed algebraic identity Section 3 found for Collatz). Consequently the Juggler sequence is a plausible, but not yet constructed, instance of this recipe: (B) holds outright, (A) is believed achievable in polynomial gate count by known general techniques for reversible square roots, but exhibiting the specific circuit, and confirming its gate count is the polynomial (or, per D.1, even just quadratic) degree this example for this appendix needs, is separate work we have not carried out.

D.4 Scope

This appendix makes no claim beyond what (A)–(B) formalise: it is a statement of exactly which two ingredients this paper’s construction needs, not a claim that those ingredients are easy to obtain for any given φ , nor a claim that a version of CC’-style logarithmic stopping time (Section 2) holds for every such recurrence ; that, too, would need its own derivation for each new φ , exactly as Section 2 required one for Collatz specifically. This appendix sketched how the main algorithm in the main text generalises to other iteration giving the juggler sequence as an example; but there is more examples than juggler sequence not given in this appendix D.